05 Automated Log Parser & SOC Threat Detector

05-soc-log-detector

Parses auth.log, syslog and Apache logs, normalises time zones to UTC and runs Sigma-style YAML rules for SSH brute force, password spraying, HTTP 401/403 bursts and impossible travel.

Verified here: 66 passed; 0 failed; 0 skipped

Measured results

- Held-out test on 30 freshly generated datasets: overall precision 0.83 (271/326 alerts), recall 0.78 (168/215 attack scenarios)
- 100% of normal-intensity attacks detected; 0 of 47 deliberately low-and-slow brute-force/spray/HTTP attacks caught at default thresholds (documented blind spot)
- Sensitive preset raises recall to 0.94 but drops precision to 0.70 (benign look-alikes alerted: 44 -> 118 of 271)
- Sample day: 6,030 log lines from 3 sources parsed in 0.0426s, all 9 deliberately malformed lines rejected and reported, 11 alerts
- 66 pytest tests pass (parsers, sliding windows, geo/Haversine, rule engine, false positives, CLI, Streamlit app)

Data provenance and caveats

- All logs are GENERATED by src/generator.py (seeded, deterministic): 24 h of auth.log (BSD syslog, Asia/Kolkata local time), syslog (ISO-8601) and Apache access.log (America/New_York offset) with labelled attack scenarios and benign look-alikes.
- All IP addresses are RFC 5737 documentation addresses (192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24).
- IP-to-location mapping is a fictional offline table (data/simulated_geo.csv); it is not geolocation.
- No real or public log dataset was used; precision/recall are measured against the generator's own labels and say nothing about real-world efficacy.
- The same author wrote the generator and the rules; thresholds were set before the held-out run and not tuned on held-out seeds, but the generator was fixed once after a held-out run exposed unlabelled cross-city office logins.
- Low-and-slow ('hard') attack variants are generated below the default thresholds on purpose to measure the blind spot; impossible-travel 'hard' cases (7/7) were still caught via the attacker-to-home return leg.
- Geo lookups are simulated; a real deployment would swap in a licensed GeoIP database.
- No Sigma converter compatibility: the YAML follows Sigma's selection/condition style plus a custom correlation block.

Project contents

README.md, requirements.txt, cli.py, app.py, src/__init__.py, src/detectors.py, src/evaluate.py, src/generator.py, src/geo.py, src/models.py, src/parsers.py, src/pipeline.py, src/report.py, src/rules.py, tests/conftest.py, tests/test_cli_and_app.py, tests/test_evaluate.py, tests/test_generator_and_fp.py, tests/test_geo.py, tests/test_parsers.py, tests/test_rules.py, tests/test_windows.py, .gitignore, config/sensitive.yml, config/settings.yml, pytest.ini, rules/http_auth_burst.yml, rules/impossible_travel.yml, rules/ssh_bruteforce.yml, rules/ssh_password_spray.yml, samples/labels.json, scripts/run_demo.sh

Automated Log Parser & SOC Threat Detector

A defensive log-analysis tool that parses Linux `auth.log` / `syslog` and Apache access logs, normalises every timestamp to UTC, and runs Sigma-style YAML rules to raise alerts for **SSH brute force, password spraying, bursts of HTTP 401/403 responses and impossible travel**. It writes JSON/CSV alerts and a static HTML report, and includes a Streamlit dashboard with a timeline. Precision and recall are measured on held-out generated data with ground-truth labels.

> **Data notice.** Every log line in this repository is **generated** by `src/generator.py`. All IP addresses are RFC 5737 documentation addresses (`192.0.2.0/24`, `198.51.100.0/24`, `203.0.113.0/24`), and the IP-to-location table in `data/simulated_geo.csv` is **fictional**. The numbers below show how the detectors behave on this synthetic data. They do not show real-world efficacy.

Problem

A SOC analyst gets thousands of authentication and web log lines a day from hosts in different time zones and formats. Most of it is noise. The few lines that matter look like noise one at a time: a password guessed 80 times in three minutes, one IP trying 17 different usernames once each, a login from 13,000 km away 100 minutes after the user logged in from home. Spotting these means parsing messy logs reliably, correlating events over time windows, and keeping thresholds tunable so false positives can be managed.

How it works / Architecture

```
 auth.log (BSD syslog, no year/zone)   syslog (ISO-8601)   access.log (Apache combined)
          |                              |                  |
          v                              v                  v
   +-------------------------------------------------------------------------+
   | parsers.py  anchored regexes, IP validation, tz -> UTC, year rollover, |
   |             invalid lines collected (never raised), 4 KB line cap      |
   +-------------------------------------------------------------------------+
          |  normalised Event(ts_utc, source, action, user, src_ip, status, ...)
          v
   +------------------------+     +---------------------------------------+
   | rules/*.yml (Sigma-ish) | --> | rules.py  selections, modifiers       |
   | + config overrides      |     |  (contains/startswith/re/cidr/gte...), |
   | + CLI --set             |     |  and/or/not condition parser          |
   +------------------------+     +---------------------------------------+
          |  matching events
          v
   +-------------------------------------------------------------------------+
   | detectors.py  event_count   (sliding window per src_ip+user)          |
   |               value_count   (distinct users per src_ip in window)     |
   |               impossible_travel (geo.py: simulated CIDR table +       |
   |                           Haversine, km / hours > max speed)          |
   +-------------------------------------------------------------------------+
          |  Alert(rule, level, entity, first/last seen, evidence file:line)
          v
   report.py: alerts.json / alerts.csv / invalid_lines.csv / timeline.png / dashboard.html
   app.py:    Streamlit dashboard (timeline, alerts, travel map, evaluation, data quality)
   evaluate.py: score alerts vs generator labels -> precision / recall with denominators
```

Detections (rules/):

Rule	Logic (default threshold)	ATT&CK

`soc-ssh-001` SSH brute force	>= 10 failed passwords for the same src_ip + user within 5 min	T1110.001
`soc-ssh-002` Password spraying	>= 8 distinct usernames failing from one src_ip within 30 min	T1110.003
`soc-web-001` HTTP 401/403 burst	>= 20 responses with status 401/403 to one client within 60 s (`/healthz` filtered)	T1110.004
`soc-geo-001` Impossible travel	consecutive successful SSH/web logins of a user, >= 500 km apart, implied speed > 900 km/h; VPN egress CIDR filtered	T1078

Once a window crosses its threshold, an alert opens. Later matching events from the same entity are folded into it while the gap stays within the window, so a burst produces one alert rather than one per line.

Security design decisions

- **Treat every input as hostile.** All regexes are anchored with bounded quantifiers. Lines over 4 KB are rejected. IPs go through `ipaddress`, timestamps are range-checked (`Feb 30`, `25:61:00` and month `13` are rejected), and bytes that aren't UTF-8 are replaced. A bad line becomes an `InvalidLine` with an 80-character printable preview. It never raises, and never copies raw junk into reports.
- **Time zones are explicit.** BSD syslog has no year and no zone, so the host's zone (`--tz`, default from `config/settings.yml`) and year are configuration. A December-to-January jump rolls the year forward, and a bad line can't corrupt that state (there's a test for this). ISO-8601 and Apache offsets are honoured. Everything is correlated in UTC. Reading the IST `auth.log` as UTC would shift events by exactly 5 h 30 min (also tested).
- **Rules are data, loaded safely.** Rules are read with `yaml.safe_load` only (a test checks that a `!!python/object` payload is refused). Thresholds can be changed in `config/settings.yml` or with `--set rule.key=value` without editing rule files. Unknown rule ids in overrides are an error, not a silent no-op.
- **Brute force is grouped by IP + user and spraying by distinct users per IP.** This keeps the two techniques separate: a sprayer trying 17 accounts once each never trips the brute-force rule.
- **Geo is simulated and the code says so.** Documentation IPs have no location. `geo.py` uses longest-prefix matching on a fictional CIDR table. Unknown locations are counted, not guessed. A licensed GeoIP database could be swapped in behind `GeoTable.lookup`.
- **Evidence is minimised.** Alerts carry `file:line` references (capped at 50) and a sample of at most 20 usernames, not raw log lines. The HTML report has no JavaScript and no external assets.
- **Threat model.** This tool is for detection after the fact on logs you already hold. It does not block anything, assumes logs weren't tampered with, and can be evaded by attackers who stay under thresholds (measured below).

Project structure

```
05-soc-log-detector/
|-- cli.py                # generate / analyze / evaluate / rules
|-- app.py                # Streamlit dashboard (5 tabs)
|-- src/
|   |-- parsers.py        # auth.log/syslog (BSD + ISO) and Apache parsers
|   |-- rules.py          # Sigma-like YAML loader, modifiers, condition parser, overrides
|   |-- detectors.py      # event_count / value_count / impossible_travel engines
|   |-- geo.py            # simulated CIDR->location table, Haversine
|   |-- generator.py      # seeded labelled log generator (RFC 5737 IPs only)
|   |-- evaluate.py       # precision/recall vs labels, multi-seed aggregation
|   |-- pipeline.py       # parse -> rules -> alerts glue
|   |-- report.py         # JSON/CSV/PNG/HTML outputs
|   `-- models.py         # Event, Alert, InvalidLine
|-- rules/                # 4 YAML detection rules
```

```
|-- config/settings.yml       # time zone, year, threshold overrides
|-- config/sensitive.yml      # lower-threshold preset used for the trade-off study
|-- data/simulated_geo.csv    # fictional IP->region table
|-- samples/                  # generated sample day (seed 42) + labels.json/csv
|-- scripts/run_demo.sh       # regenerates everything in outputs/
|-- tests/                    # 66 pytest tests
`-- outputs/                  # alerts, reports, charts, evaluation, test log
```

Setup & how to run

```bash
pip install -r requirements.txt

python cli.py generate --seed 42 --out samples          # labelled sample day
python cli.py analyze  --logs samples/logs --out outputs  # alerts + reports (+ score vs
labels)
python cli.py analyze  --logs samples/logs --set soc-ssh-001.threshold=15 --set soc-
web-001.timespan=120s
python cli.py analyze  --logs /path/to/logs --tz Asia/Kolkata --year 2026 --fail-on-alert
  # exit 1 on alerts
python cli.py evaluate --seeds 1001-1030 --compare config/sensitive.yml --out outputs
python cli.py rules                                     # effective thresholds

pytest                      # 66 tests
streamlit run app.py        # dashboard; sidebar sliders re-run detection live
bash scripts/run_demo.sh    # rebuild all outputs (about 15 s)
```

`analyze` picks up `*.log`, `syslog`, `messages`, `secure` and `*access*` files in the folder and detects the format of each one automatically.

Results (real numbers from this run)

Tests: 66 passed, 0 failed, in 3.4 s (`outputs/pytest_output.txt`).

Sample day (seed 42): 6,030 lines from 3 files gave 6,021 events and 9 invalid lines. That is exactly the 9 malformed lines the generator injected, each rejected with a reason. Parsing took 0.0426 s and detection 0.0197 s. There were 11 alerts. Scored against this sample's labels, every normal-intensity attack was found (2/2 brute force, 1/1 spray, 1/1 burst, 1/1 travel). The slow HTTP stuffing attack (41 requests at about one every 5 s) was missed. Two alerts were false positives on benign look-alikes: a service account looping on an expired password, and a web client retrying an expired token.

Held-out evaluation: 30 freshly generated datasets (seeds 1001-1030, none of them used while writing the rules). They contain 215 labelled attack scenarios and 271 benign look-alike scenarios. An alert counts as a true positive only if it matches a malicious scenario of the same class, the same entity and an overlapping time window (+/-5 min).

Default thresholds:

Detector	TP / alerts	Precision	Detected / scenarios	Recall	Normal caught	Low-and-slow caught
brute_force	122/130	**0.94**	57/77	**0.74**	57/57	0/20
password_spray	36/43	**0.84**	36/48	**0.75**	36/36	0/12
http_auth_burst	33/47	**0.70**	33/48	**0.69**	33/33	0/15
impossible_travel	80/106	**0.75**	42/42	**1.00**	35/35	7/7

Overall: precision **0.83** (271/326), recall **0.78** (168/215). 44 of 271 benign

scenarios raised an alert. All false positives came from the deliberately ambiguous look-alikes: password-rotation morning behind the office NAT, a stale service-account password, mobile-carrier NAT and a token-retry loop.

Sensitive preset (`config/sensitive.yml`: brute force 5 in 60 min, spray 5 users in 2 h, HTTP 10 in 60 s, max speed 700 km/h):

Detector	TP / alerts	Precision	Detected / scenarios	Recall	Normal caught	Low-and-slow caught
brute_force	135/179	**0.75**	65/77	**0.84**	57/57	8/20
password_spray	48/93	**0.52**	48/48	**1.00**	36/36	12/12
http_auth_burst	48/62	**0.77**	48/48	**1.00**	33/33	15/15
impossible_travel	82/112	**0.73**	42/42	**1.00**	35/35	7/7

Overall: precision **0.70**, recall **0.94**. 118 of 271 benign scenarios alerted. This is the usual SOC trade-off: catching low-and-slow attacks costs a lot more analyst noise.




Sample output

```
$ python cli.py generate --seed 42 --out samples
generated seed=42 into samples/logs
  auth.log       702 lines (2 deliberately malformed)
  syslog         402 lines (3 deliberately malformed)
  access.log    4926 lines (4 deliberately malformed)
  scenarios: 6 malicious, 9 benign look-alikes
    S001 benign benign_typo_burst           normal 6 failed passwords then success (below brute-force threshold)
    S002 benign benign_nat_typos            normal 5 distinct users fail once each via shared office NAT
    S003 benign benign_legit_travel         normal ~7,700 km in 11+ h (< 900 km/h): plausible flight
    S004 benign benign_vpn_switch           normal VPN egress is allowlisted in the impossible-travel rule
    S005 benign benign_web_typos            normal 6 x 401 in < 1 min (below 20/60 s)
    S006 benign benign_crawler              normal 250 crawler requests, ~4% 403 spread over ~10 min
    S007 benign benign_healthcheck_403      normal 60 x 403 on /healthz in 30 s; excluded by filter_healthcheck
    S008 benign benign_stale_service_account normal service account retrying an expired password every 15-45 s
    S009 benign benign_token_retry_loop     normal buggy client retrying an expired token (bait for HTTP burst rule)
    S010 ATTACK brute_force                 normal 93 attempts on ['ubuntu', 'root', 'sneha']
    S011 ATTACK brute_force                 normal 109 attempts on ['admin', 'nikhil']
    S012 ATTACK password_spray              normal 17 usernames
    S013 ATTACK http_auth_burst             normal 85 x forced_browsing
    S014 ATTACK http_auth_burst             hard   41 x stuffing (slow)
    S015 ATTACK impossible_travel           normal stolen-credential login via ssh from SIM-US-SFO after 1.7 h
```

```
$ python cli.py analyze --logs samples/logs --out outputs
parsed 6021 events from 3 files (6030 lines, 9 invalid) in 0.04s; detection 0.020s
```

```
time zone for BSD syslog headers: Asia/Kolkata; all output timestamps are UTC
11 alerts: soc-geo-001=2, soc-ssh-001=6, soc-ssh-002=1, soc-web-001=2
  [HIGH  ] 2026-10-19 20:43:31Z soc-geo-001 user rahul: Simulated Hyderabad -> Simulated
San Francisco, 13,504 km in 103 min (7,898 km/h; limit 900 km/h; simulated geo)
  [HIGH  ] 2026-10-19 20:57:13Z soc-ssh-001 17 matching events from src_ip=198.51.100.10,
user=svc-backup over 416s (threshold 10 within 0:05:00)
  [HIGH  ] 2026-10-19 22:26:06Z soc-geo-001 user rahul: Simulated San Francisco ->
Simulated Hyderabad, 13,504 km in 354 min (2,291 km/h; limit 900 km/h; simulated geo)
  [HIGH  ] 2026-10-20 00:35:47Z soc-ssh-001 55 matching events from src_ip=203.0.113.20,
user=nikhil over 235s (threshold 10 within 0:05:00)
  [HIGH  ] 2026-10-20 00:35:49Z soc-ssh-001 54 matching events from src_ip=203.0.113.20,
user=admin over 237s (threshold 10 within 0:05:00)
  [HIGH  ] 2026-10-20 01:07:24Z soc-ssh-002 17 distinct user values in 24 events from
src_ip=203.0.113.166 over 19 min (threshold 8 within 0:30:00)
  [MEDIUM] 2026-10-20 07:48:58Z soc-web-001 104 matching events from src_ip=192.0.2.18
over 205s (threshold 20 within 0:01:00)
  [HIGH  ] 2026-10-20 09:39:58Z soc-ssh-001 35 matching events from src_ip=203.0.113.142,
user=root over 196s (threshold 10 within 0:05:00)
  [HIGH  ] 2026-10-20 09:40:02Z soc-ssh-001 35 matching events from src_ip=203.0.113.142,
user=sneha over 188s (threshold 10 within 0:05:00)
  [HIGH  ] 2026-10-20 09:40:07Z soc-ssh-001 23 matching events from src_ip=203.0.113.142,
user=ubuntu over 184s (threshold 10 within 0:05:00)
  [MEDIUM] 2026-10-20 15:33:38Z soc-web-001 85 matching events from src_ip=203.0.113.43
over 72s (threshold 20 within 0:01:00)
score vs this sample's labels (TP alerts/alerts, detected/scenarios):
  brute_force        5/6  2/2
  password_spray     1/1  1/1
  http_auth_burst    1/2  1/2
  impossible_travel  2/2  1/1
wrote outputs/alerts.json, alerts.csv, invalid_lines.csv, timeline.png, dashboard.html
```

```
$ python cli.py evaluate --seeds 1001-1030 --compare config/sensitive.yml --out outputs
== default: 30 held-out datasets (seeds 1001-1030)
  detector            TP/alerts precision  detected/sc  recall recall(normal) hard caught
  brute_force         122/130      0.9385    57/77      0.7403           1.0     0/20
  password_spray       36/43       0.8372    36/48       0.75            1.0     0/12
  http_auth_burst      33/47       0.7021    33/48      0.6875           1.0     0/15
  impossible_travel    80/106      0.7547    42/42        1.0            1.0     7/7
  overall: precision 0.8313 (271/326), recall 0.7814 (168/215); benign look-alike
scenarios alerted: 44/271
== sensitive: 30 held-out datasets (seeds 1001-1030)
  detector            TP/alerts precision  detected/sc  recall recall(normal) hard caught
  brute_force         135/179      0.7542    65/77      0.8442           1.0     8/20
  password_spray       48/93       0.5161    48/48        1.0            1.0    12/12
  http_auth_burst      48/62       0.7742    48/48        1.0            1.0    15/15
  impossible_travel    82/112      0.7321    42/42        1.0            1.0     7/7
  overall: precision 0.7018 (313/446), recall 0.9442 (203/215); benign look-alike
scenarios alerted: 118/271
```

Limitations & future work

- **Synthetic data only.** The same author wrote the generator and the rules. Thresholds were set before the held-out run and not tuned on it, but the generator was corrected once after a held-out run exposed unlabelled cross-city office logins. Real logs will be messier. A next step is to validate on public datasets such as loghub's OpenSSH logs or on a honeypot.
- **Simulated geo.** Distances come from a fictional table. Real GeoIP has errors, and

VPNs, mobile carriers and cloud egress cause false positives that need allowlists or ASN context.
- **Threshold evasion.** At default thresholds, 0 of 47 low-and-slow brute-force, spray and HTTP attacks were caught. Longer windows, per-account baselines or risk scoring would help.
- **Rule language.** The YAML borrows Sigma's selection/condition style and adds a custom `correlation` block. It is not Sigma-converter compatible.
- **Batch processing.** Files are analysed in batch. Streaming (tail -f, journald, syslog over TCP), state persistence and alert deduplication across runs are future work.
- **Formats.** Only OpenSSH messages, BSD/ISO syslog headers and Apache combined/common are parsed. Other formats (journald JSON, Windows events) would need new parsers.

Resume bullets

- Built a Python SOC log analyser that parses auth.log, syslog and Apache logs across time zones into UTC. It processed 6,030 lines in 0.0426 s and rejected all 9 injected malformed lines without crashing.
- Designed a Sigma-style YAML rule engine (field modifiers, boolean condition parser, configurable sliding-window thresholds) with detections for SSH brute force, password spraying, HTTP 401/403 bursts and impossible travel (Haversine on a simulated geo table).
- Measured detection quality on 30 held-out labelled synthetic datasets: 0.83 precision and 0.78 recall (168/215 attacks). Quantified the threshold trade-off: a sensitive preset raised recall to 0.94 and lowered precision to 0.70.
- Shipped a CLI, JSON/CSV/HTML reporting and a Streamlit dashboard with an alert timeline and live threshold tuning, covered by 66 pytest tests (parsers, windows, false positives, geo, CLI, UI).

Automated Log Parser & SOC Threat Detector — ACTUAL SAMPLE OUTPUT

```
$ python cli.py generate --seed 42 --out samples
generated seed=42 into samples/logs
  auth.log       702 lines (2 deliberately malformed)
  syslog         402 lines (3 deliberately malformed)
  access.log    4926 lines (4 deliberately malformed)
  scenarios: 6 malicious, 9 benign look-alikes
    S001 benign benign_typo_burst            normal 6 failed passwords then success (below brute-force threshold)
    S002 benign benign_nat_typos             normal 5 distinct users fail once each via shared office NAT
    S003 benign benign_legit_travel          normal ~7,700 km in 11+ h (< 900 km/h): plausible flight
    S004 benign benign_vpn_switch            normal VPN egress is allowlisted in the impossible-travel rule
    S005 benign benign_web_typos             normal 6 x 401 in < 1 min (below 20/60 s)
    S006 benign benign_crawler               normal 250 crawler requests, ~4% 403 spread over ~10 min
    S007 benign benign_healthcheck_403       normal 60 x 403 on /healthz in 30 s; excluded by filter_healthcheck
    S008 benign benign_stale_service_account normal service account retrying an expired password every 15-45 s
    S009 benign benign_token_retry_loop      normal buggy client retrying an expired token (bait for HTTP burst rule)
    S010 ATTACK brute_force                  normal 93 attempts on ['ubuntu', 'root', 'sneha']
    S011 ATTACK brute_force                  normal 109 attempts on ['admin', 'nikhil']
    S012 ATTACK password_spray               normal 17 usernames
    S013 ATTACK http_auth_burst              normal 85 x forced_browsing
    S014 ATTACK http_auth_burst              hard   41 x stuffing (slow)
    S015 ATTACK impossible_travel            normal stolen-credential login via ssh from SIM-US-SFO after 1.7 h
  labels: samples/labels.json, samples/labels.csv

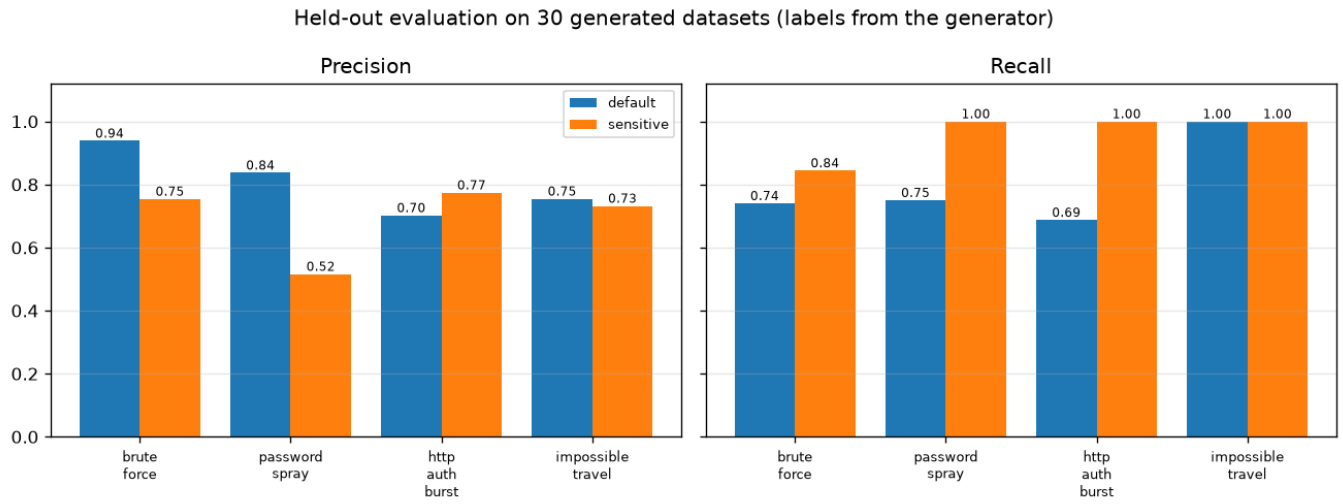
$ python cli.py rules
soc-web-001  [medium]  Burst of HTTP 401/403 responses from one client
    {'type': 'event_count', 'group_by': ['src_ip'], 'timespan': '60s', 'threshold': 20}
    tags: attack.credential_access, attack.t1110.004
soc-geo-001  [high]  Impossible travel between two successful logins
    {'type': 'impossible_travel', 'group_by': ['user'], 'max_speed_kmh': 900, 'min_distance_km': 500}
    tags: attack.initial_access, attack.t1078
soc-ssh-001  [high]  SSH brute force from a single source IP
    {'type': 'event_count', 'group_by': ['src_ip', 'user'], 'timespan': '5m', 'threshold': 10}
    tags: attack.credential_access, attack.t1110.001
soc-ssh-002  [high]  SSH password spraying (many usernames from one source)
    {'type': 'value_count', 'field': 'user', 'group_by': ['src_ip'], 'timespan': '30m', 'threshold': 8}
    tags: attack.credential_access, attack.t1110.003

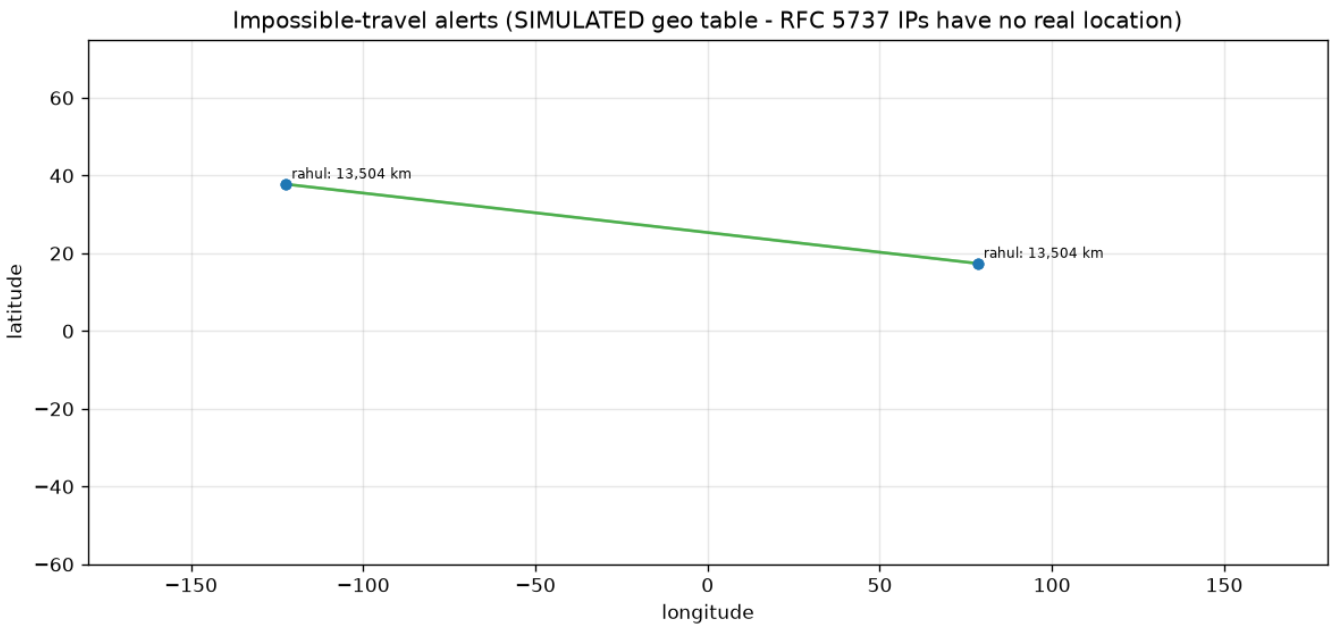
$ python cli.py evaluate --seeds 1001-1030 --compare config/sensitive.yml --out outputs
== default: 30 held-out datasets (seeds 1001-1030)
  detector            TP/alerts precision  detected/sc  recall recall(normal) hard caught
  brute_force         122/130      0.9385    57/77      0.7403            1.0     0/20
  password_spray       36/43       0.8372    36/48        0.75            1.0     0/12
  http_auth_burst      33/47       0.7021    33/48      0.6875            1.0     0/15
  impossible_travel    80/106      0.7547    42/42         1.0            1.0     7/7
  overall: precision 0.8313 (271/326), recall 0.7814 (168/215); benign look-alike scenarios alerted: 44/271
== sensitive: 30 held-out datasets (seeds 1001-1030)
  detector            TP/alerts precision  detected/sc  recall recall(normal) hard caught
  brute_force         135/179      0.7542    65/77      0.8442            1.0     8/20
  password_spray       48/93       0.5161    48/48         1.0            1.0    12/12
  http_auth_burst      48/62       0.7742    48/48         1.0            1.0    15/15
  impossible_travel    82/112      0.7321    42/42         1.0            1.0     7/7
  overall: precision 0.7018 (313/446), recall 0.9442 (203/215); benign look-alike scenarios alerted: 118/271
```

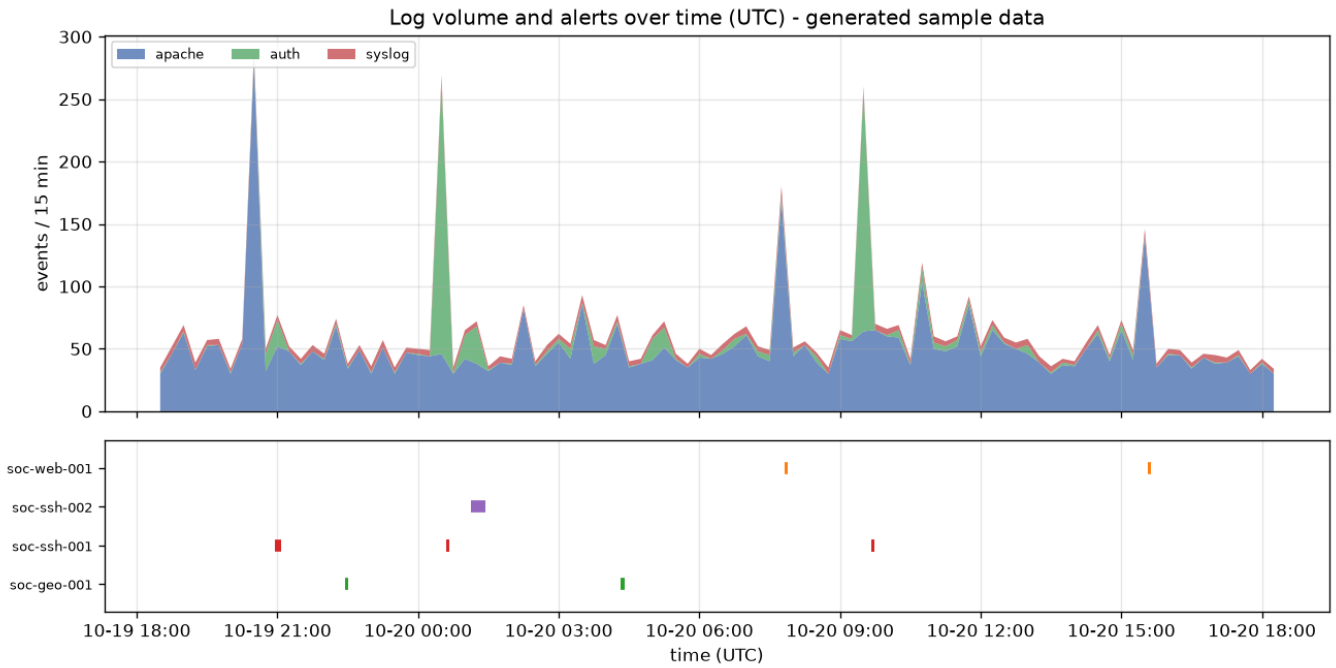
```
evaluation took 6.4s
wrote outputs/evaluation.json, outputs/evaluation.png

$ python cli.py analyze --logs samples/logs --out outputs
parsed 6021 events from 3 files (6030 lines, 9 invalid) in 0.04s; detection 0.020s
time zone for BSD syslog headers: Asia/Kolkata; all output timestamps are UTC
11 alerts: soc-geo-001=2, soc-ssh-001=6, soc-ssh-002=1, soc-web-001=2
  [HIGH  ] 2026-10-19 20:43:31Z soc-geo-001 user rahul: Simulated Hyderabad -> Simulated San Francisco, 13,504 km in 103 min (7,898 km/h; limit 900 km/h; simulated geo)
  [HIGH  ] 2026-10-19 20:57:13Z soc-ssh-001 17 matching events from src_ip=198.51.100.10, user=svc-backup over 416s (threshold 10 within 0:05:00)
  [HIGH  ] 2026-10-19 22:26:06Z soc-geo-001 user rahul: Simulated San Francisco -> Simulated Hyderabad, 13,504 km in 354 min (2,291 km/h; limit 900 km/h; simulated geo)
  [HIGH  ] 2026-10-20 00:35:47Z soc-ssh-001 55 matching events from src_ip=203.0.113.20, user=nikhil over 235s (threshold 10 within 0:05:00)
  [HIGH  ] 2026-10-20 00:35:49Z soc-ssh-001 54 matching events from src_ip=203.0.113.20, user=admin over 237s (threshold 10 within 0:05:00)
  [HIGH  ] 2026-10-20 01:07:24Z soc-ssh-002 17 distinct user values in 24 events from src_ip=203.0.113.166 over 19 min (threshold 8 within 0:30:00)
  [MEDIUM] 2026-10-20 07:48:58Z soc-web-001 104 matching events from src_ip=192.0.2.18 over 205s (threshold 20 within 0:01:00)
  [HIGH  ] 2026-10-20 09:39:58Z soc-ssh-001 35 matching events from src_ip=203.0.113.142, user=root over 196s (threshold 10 within 0:05:00)
  [HIGH  ] 2026-10-20 09:40:02Z soc-ssh-001 35 matching events from src_ip=203.0.113.142, user=sneha over 188s (threshold 10 within 0:05:00)
  [HIGH  ] 2026-10-20 09:40:07Z soc-ssh-001 23 matching events from src_ip=203.0.113.142, user=ubuntu over 184s (threshold 10 within 0:05:00)
  [MEDIUM] 2026-10-20 15:33:38Z soc-web-001 85 matching events from src_ip=203.0.113.43 over 72s (threshold 20 within 0:01:00)
score vs this sample's labels (TP alerts/alerts, detected/scenarios):
  brute_force        5/6  2/2
  password_spray     1/1  1/1
  http_auth_burst    1/2  1/2
  impossible_travel  2/2  1/1
wrote outputs/alerts.json, alerts.csv, invalid_lines.csv, timeline.png, dashboard.html

```


Held-out evaluation on 30 generated datasets (labels from the generator)
Precision
Recall
default
sensitive
1.0
0.8
0.6
0.4
0.2
0.0
0.94
0.75
0.84
0.52
0.70
0.77
0.75
0.73
0.74
0.84
0.75
1.00
0.69
1.00
1.00
1.00
brute force
password spray
http auth burst
impossible travel


Impossible-travel alerts (SIMULATED geo table - RFC 5737 IPs have no real location)
rahul: 13,504 km
rahul: 13,504 km
latitude
longitude
60
40
20
0
−20
−40
−60
−150
−100
−50
0
50
100
150


Log volume and alerts over time (UTC) - generated sample data
apache
auth
syslog
events / 15 min
300
250
200
150
100
50
0
soc-web-001
soc-ssh-002
soc-ssh-001
soc-geo-001
10-19 18:00
10-19 21:00
10-20 00:00
10-20 03:00
10-20 06:00
10-20 09:00
10-20 12:00
10-20 15:00
10-20 18:00
time (UTC)

05-soc-log-detector/requirements.txt

```
pyyaml>=6.0
pandas>=2.0
matplotlib>=3.7
plotly>=5.18
streamlit>=1.40
pytest>=7.4

```

```
#!/usr/bin/env python3
"""Command-line interface for the SOC log detector.

    python cli.py generate --seed 42 --out samples
    python cli.py analyze  --logs samples/logs --out outputs
    python cli.py evaluate --seeds 1001-1030
    python cli.py rules
"""
from __future__ import annotations

import argparse
import json
import sys
import time
from pathlib import Path

import yaml

from src.evaluate import evaluate_seeds, score
from src.generator import generate
from src.pipeline import analyze_dir
from src.report import (evaluation_chart, html_dashboard, timeline_chart, travel_chart,
                        write_alerts_csv, write_alerts_json, write_invalid_csv)
from src.rules import RuleError, load_rules, parse_set_args

ROOT = Path(__file__).resolve().parent


def _settings(path: str | None) -> dict:
    p = Path(path) if path else ROOT / "config" / "settings.yml"
    return (yaml.safe_load(p.read_text()) or {}) if p.exists() else {}


def _seed_range(text: str) -> list[int]:
    if "-" in text:
        a, b = (int(x) for x in text.split("-", 1))
        if b < a or b - a > 1000:
            raise argparse.ArgumentTypeError("seed range must be ascending and <= 1000 seeds")
        return list(range(a, b + 1))
    return [int(x) for x in text.split(",")]


def _overrides(args, settings: dict) -> dict:
    ov = dict(settings.get("overrides") or {})
    if getattr(args, "overrides", None):
        for rid, vals in (yaml.safe_load(Path(args.overrides).read_text()) or {}).items():
            ov[rid] = {**ov.get(rid, {}), **vals}
    for rid, vals in parse_set_args(getattr(args, "set", None) or []).items():
        ov[rid] = {**ov.get(rid, {}), **vals}
    return ov


def cmd_generate(args) -> int:
    doc = generate(args.out, seed=args.seed)
    mal = [s for s in doc["scenarios"] if s["malicious"]]
    print(f"generated seed={args.seed} into {args.out}/logs")
    for f, c in doc["meta"]["files"].items():
        print(f"  {f:<11} {c['lines']:>6} lines ({c['invalid_injected']} deliberately malformed)")
    print(f"  scenarios: {len(mal)} malicious, {len(doc['scenarios']) - len(mal)} benign look-alik
es")
    for s in doc["scenarios"]:
        tag = "ATTACK" if s["malicious"] else "benign"
        print(f"    {s['scenario_id']} {tag:<6} {s['type']:<28} {s['difficulty']:<6} {s['note']}")
    print(f"  labels: {args.out}/labels.json, {args.out}/labels.csv")
    return 0
```

```
65
66
67  def cmd_analyze(args) -> int:
68      st = _settings(args.config)
69      tz = args.tz or st.get("auth_log_timezone", "UTC")
70      year = args.year or st.get("log_year", 2026)
71      out = Path(args.out)
72      out.mkdir(parents=True, exist_ok=True)
73      run = analyze_dir(args.logs, args.rules, tz=tz, year=year, overrides=_overrides(args, st),
74                        tz_overrides=st.get("timezone_overrides") or {})
75      s, alerts = run["summary"], run["alerts"]
76      s["auth_log_timezone"], s["log_year"] = tz, year
77      s["thresholds"] = {r.rule_id: {k: v for k, v in r.correlation.items() if k != "group_by"}
78                         for r in run["rules"]}
79      write_alerts_json(alerts, s, out / "alerts.json")
80      write_alerts_csv(alerts, out / "alerts.csv")
81      write_invalid_csv(run["invalid"], out / "invalid_lines.csv")
82      timeline_chart(run["events"], alerts, out / "timeline.png")
83      travel_chart(alerts, out / "impossible_travel.png")
84      labels_path = Path(args.labels) if args.labels else Path(args.logs).parent / "labels.json"
85      if labels_path.exists():
86          sc = score(alerts, json.loads(labels_path.read_text())["scenarios"])
87          (out / "sample_score.json").write_text(json.dumps(sc, indent=2))
88          s["sample_score"] = {k: {x: v[x] for x in ("alerts", "tp_alerts", "scenarios", "detected")
    }
89                               for k, v in sc["per_label"].items()}
90      ev_path = out / "evaluation.json"
91      ev = json.loads(ev_path.read_text())["default"] if ev_path.exists() else None
92      html_dashboard(alerts, s, out / "timeline.png", out / "dashboard.html", ev)
93      print(f"parsed {s['events']} events from {len(run['files'])} files "
94            f"({s['lines_read']} lines, {s['invalid_lines']} invalid) "
95            f"in {s['parse_seconds']:.2f}s; detection {s['detect_seconds']:.3f}s")
96      print(f"time zone for BSD syslog headers: {tz}; all output timestamps are UTC")
97      print(f"{len(alerts)} alerts: " + ", ".join(f"{k}={v}" for k, v in s["alerts_by_rule"].items()
    ))
98      for a in alerts:
99          print(f"  [{a.level.upper():<6}] {a.first_seen:%Y-%m-%d %H:%M:%S}Z {a.rule_id} {a.message}
    ")
100     if "sample_score" in s:
101         print("score vs this sample's labels (TP alerts/alerts, detected/scenarios):")
102         for k, v in s["sample_score"].items():
103             print(f"  {k:<18} {v['tp_alerts']}/{v['alerts']}  {v['detected']}/{v['scenarios']}")
104     print(f"wrote {out}/alerts.json, alerts.csv, invalid_lines.csv, timeline.png, dashboard.html")
105     return 1 if (args.fail_on_alert and alerts) else 0
106
107
108 def cmd_evaluate(args) -> int:
109     st = _settings(args.config)
110     t0 = time.perf_counter()
111     presets = {"default": _overrides(argparse.Namespace(overrides=None, set=args.set), st)}
112     if args.compare:
113         presets["sensitive"] = _overrides(argparse.Namespace(overrides=args.compare, set=args.set)
    , st)
114     results = {name: evaluate_seeds(args.seeds, args.rules, ov) for name, ov in presets.items()}
115     elapsed = time.perf_counter() - t0
116     for name, ev in results.items():
117         print(f"== {name}: {ev['datasets']} held-out datasets (seeds {ev['seeds'][0]}-{ev['seeds']
    [1]})")
118         print(f"  {'detector':<18} {'TP/alerts':>10} {'precision':>9} {'detected/sc':>12} {'recall
    ':>7} "
119               f"{'recall(normal)':>14} {'hard caught':>11}")
120         for lab, r in ev["per_label"].items():
121             print(f"  {lab:<18} {r['tp_alerts']:>4}/{r['alerts']:<5} {r['precision']!s:>9} "
122                   f"{r['detected']:>5}/{r['scenarios']:<6} {r['recall']!s:>7} {r['recall_normal']!
    s:>14} "
```

```
                  f"{r['detected_hard']:>5}/{r['scenarios_hard']:<5}")
        o = ev["overall"]
        print(f"  overall: precision {o['precision']} ({o['tp_alerts']}/{o['alerts']}), "
              f"recall {o['recall']} ({o['detected']}/{o['scenarios']}); benign look-alike "
              f"scenarios alerted: {ev['benign_scenarios_alerted']}/{ev['benign_scenarios']}")
    print(f"evaluation took {elapsed:.1f}s")
    if args.out:
        out = Path(args.out)
        out.mkdir(parents=True, exist_ok=True)
        (out / "evaluation.json").write_text(json.dumps(results, indent=2))
        evaluation_chart(results, out / "evaluation.png")
        print(f"wrote {out}/evaluation.json, {out}/evaluation.png")
    return 0


def cmd_rules(args) -> int:
    for r in load_rules(args.rules, _overrides(args, _settings(args.config))):
        corr = {k: v for k, v in r.correlation.items()}
        print(f"{r.rule_id}  [{r.level}]  {r.title}\n    {corr}\n    tags: {', '.join(r.tags)}")
    return 0


def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(prog="soc-detect", description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    p.add_argument("--config", help="settings YAML (default config/settings.yml)")
    sub = p.add_subparsers(dest="cmd", required=True)

    g = sub.add_parser("generate", help="write a seeded, labelled sample dataset")
    g.add_argument("--seed", type=int, default=42)
    g.add_argument("--out", default="samples")
    g.set_defaults(func=cmd_generate)

    def rule_args(sp):
        sp.add_argument("--rules", default=str(ROOT / "rules"))
        sp.add_argument("--set", action="append", metavar="RULE.KEY=VALUE",
                        help="override a rule setting, e.g. soc-ssh-001.threshold=15")
        sp.add_argument("--overrides", help="YAML file of {rule_id: {key: value}} overrides")

    a = sub.add_parser("analyze", help="parse logs, run rules, write alerts and reports")
    a.add_argument("--logs", default="samples/logs")
    a.add_argument("--out", default="outputs")
    a.add_argument("--labels", help="labels.json to score against (default: next to logs)")
    a.add_argument("--tz", help="IANA zone for BSD syslog timestamps (e.g. Asia/Kolkata)")
    a.add_argument("--year", type=int, help="year of the first BSD syslog line")
    a.add_argument("--fail-on-alert", action="store_true", help="exit 1 when any alert fires")
    rule_args(a)
    a.set_defaults(func=cmd_analyze)

    e = sub.add_parser("evaluate", help="precision/recall on freshly generated held-out datasets")
    e.add_argument("--seeds", type=_seed_range, default=_seed_range("1001-1030"))
    e.add_argument("--compare", help="second overrides YAML to compare (e.g. config/sensitive.yml)
")
    e.add_argument("--out", help="directory for evaluation.json / evaluation.png")
    rule_args(e)
    e.set_defaults(func=cmd_evaluate)

    r = sub.add_parser("rules", help="list loaded rules with effective thresholds")
    rule_args(r)
    r.set_defaults(func=cmd_rules)
    return p


def main(argv=None) -> int:
    args = build_parser().parse_args(argv)
```

```
    try:
        return args.func(args)
    except (RuleError, FileNotFoundError) as exc:
        print(f"error: {exc}", file=sys.stderr)
        return 2


if __name__ == "__main__":
    sys.exit(main())

```

```
"""Streamlit SOC dashboard: `streamlit run app.py`.

Runs fully offline on the bundled generated sample (samples/logs). Threshold sliders
re-run the detectors live so you can see the precision/recall trade-off.
"""
from __future__ import annotations

import json
from pathlib import Path

import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import streamlit as st

from src.evaluate import score
from src.pipeline import analyze_dir
from src.report import alerts_frame, events_frame

ROOT = Path(__file__).resolve().parent
LOGS = ROOT / "samples" / "logs"
LABELS = ROOT / "samples" / "labels.json"
EVAL = ROOT / "outputs" / "evaluation.json"

st.set_page_config(page_title="SOC Log Detector", layout="wide")
st.title("SOC Log Parser & Threat Detector")
st.caption("Generated sample logs only - every IP is an RFC 5737 documentation address and the "
           "IP-to-location table is fictional. Not a measure of real-world efficacy.")

with st.sidebar:
    st.header("Detection thresholds")
    bf = st.slider("SSH brute force: failures per IP+user in 5 min", 3, 30, 10)
    sp = st.slider("Password spray: distinct users per IP in 30 min", 3, 20, 8)
    hb = st.slider("HTTP 401/403 burst: responses per IP in 60 s", 5, 60, 20)
    speed = st.slider("Impossible travel: max speed (km/h)", 300, 2000, 900, step=50)
    tz = st.text_input("Time zone of auth.log (BSD syslog has none)", "Asia/Kolkata")

overrides = {"soc-ssh-001": {"threshold": bf}, "soc-ssh-002": {"threshold": sp},
             "soc-web-001": {"threshold": hb}, "soc-geo-001": {"max_speed_kmh": speed}}


@st.cache_resource(show_spinner=False)  # results hold compiled rules (not picklable)
def run(overrides_json: str, tz: str):
    res = analyze_dir(LOGS, ROOT / "rules", tz=tz, overrides=json.loads(overrides_json))
    labels = json.loads(LABELS.read_text())["scenarios"] if LABELS.exists() else []
    sc = score(res["alerts"], labels) if labels else None
    return res, labels, sc


try:
    res, labels, sc = run(json.dumps(overrides, sort_keys=True), tz)
except Exception as exc:  # e.g. an invalid time zone typed in the sidebar
    st.error(f"Could not analyse the sample logs: {exc}")
    st.stop()

alerts = res["alerts"]
adf = alerts_frame(alerts)
edf = events_frame(res["events"])
summ = res["summary"]

tab_over, tab_alerts, tab_travel, tab_eval, tab_quality = st.tabs(
    ["Overview & timeline", "Alerts", "Impossible travel", "Evaluation", "Data quality"])

with tab_over:
    c = st.columns(5)
```

```
    c[0].metric("Lines read", f"{summ['lines_read']:,}")
    c[1].metric("Events parsed", f"{summ['events']:,}")
    c[2].metric("Invalid lines", summ["invalid_lines"])
    c[3].metric("Alerts", len(alerts))
    c[4].metric("High severity", int((adf["level"] == "high").sum()) if not adf.empty else 0)
    vol = (edf.set_index("ts").groupby("source").resample("15min").size()
           .rename("events").reset_index())
    fig = px.area(vol, x="ts", y="events", color="source",
                  title="Event volume per 15 minutes (UTC)")
    st.plotly_chart(fig, width="stretch")
    if not adf.empty:
        tl = adf.copy()
        tl["end"] = tl.apply(lambda r: max(r["last_seen"], r["first_seen"] + pd.Timedelta(minutes=
3)), axis=1)
        fig2 = px.timeline(tl, x_start="first_seen", x_end="end", y="rule_id", color="level",
                           hover_data=["entity", "event_count", "message"],
                           color_discrete_map={"high": "#d62728", "medium": "#ff7f0e", "low": "#1f
77b4"},
                           title="Alert timeline")
        st.plotly_chart(fig2, width="stretch")

with tab_alerts:
    if adf.empty:
        st.info("No alerts with the current thresholds.")
    else:
        rules = st.multiselect("Rules", sorted(adf["rule_id"].unique()), default=sorted(adf["rule_
id"].unique()))
        view = adf[adf["rule_id"].isin(rules)]
        if sc:
            verdicts = {r["alert_id"]: r["verdict"] for r in sc["alerts"]}
            view = view.assign(verdict_vs_labels=view["alert_id"].map(verdicts))
        st.dataframe(view, width="stretch", hide_index=True)
        st.download_button("Download alerts (CSV)", view.to_csv(index=False), "alerts.csv", "text/
csv")
        counts = adf.groupby(["rule_id", "level"]).size().rename("alerts").reset_index()
        st.plotly_chart(px.bar(counts, x="rule_id", y="alerts", color="level", title="Alerts by ru
le"),
                        width="stretch")

with tab_travel:
    legs = [a for a in alerts if a.label == "impossible_travel"]
    st.write("Locations come from the **simulated** offline table `data/simulated_geo.csv`; "
             "documentation IPs have no real location.")
    if not legs:
        st.info("No impossible-travel alerts with the current max speed.")
    else:
        fig = go.Figure()
        for a in legs:
            d = a.details
            fig.add_trace(go.Scattergeo(lon=[d["from_lon"], d["to_lon"]], lat=[d["from_lat"], d["t
o_lat"]],
                                        mode="lines+markers", name=f'{a.group["user"]}',
                                        text=[d["from_label"], d["to_label"]]))
        fig.update_layout(title="Impossible-travel legs (simulated geo)", height=450)
        st.plotly_chart(fig, width="stretch")
        st.dataframe(pd.DataFrame([{"user": a.group["user"], **{k: a.details[k] for k in (
            "from_ip", "to_ip", "from_label", "to_label", "distance_km", "hours", "speed_kmh")}}
            for a in legs]), width="stretch", hide_index=True)

with tab_eval:
    if sc:
        st.subheader("This sample vs its generated labels (current thresholds)")
        st.dataframe(pd.DataFrame(sc["per_label"]).T[["alerts", "tp_alerts", "fp_alerts", "precisi
on",
                                                      "scenarios", "detected", "recall"]],
```

```
                    width="stretch")
        if sc["missed"]:
            st.write("Missed scenarios:", pd.DataFrame(sc["missed"]))
    if EVAL.exists():
        ev = json.loads(EVAL.read_text())
        st.subheader("Held-out evaluation (fresh seeds, saved by `cli.py evaluate`)")
        rows = []
        for preset, data in ev.items():
            for lab, r in data["per_label"].items():
                rows.append({"preset": preset, "detector": lab, "precision": r["precision"],
                             "recall": r["recall"], "TP/alerts": f'{r["tp_alerts"]}/{r["alerts"]}'
,
                             "detected/scenarios": f'{r["detected"]}/{r["scenarios"]}'})
        edf2 = pd.DataFrame(rows)
        st.dataframe(edf2, width="stretch", hide_index=True)
        st.plotly_chart(px.bar(edf2.melt(id_vars=["preset", "detector"], value_vars=["precision",
"recall"]),
                               x="detector", y="value", color="preset", barmode="group", facet_col
="variable",
                               title="Precision / recall by threshold preset"), width="stretch")

with tab_quality:
    st.write(f"Parsed **{summ['events']:,}** events; **{summ['invalid_lines']}** lines were reject
ed "
             "and kept for review (never crash on bad input).")
    inv = pd.DataFrame([vars(i) for i in res["invalid"]])
    if not inv.empty:
        st.dataframe(inv, width="stretch", hide_index=True)
    st.bar_chart(pd.Series(summ["events_by_action"], name="events"))

```

05-soc-log-detector/src/__init__.py

```
"""SOC log detector: parsers, a Sigma-like rule engine and correlation detectors
for SSH brute force, password spraying, HTTP 401/403 bursts and impossible travel.

All bundled log data is generated (see src/generator.py) and every IP address is an
RFC 5737 documentation address. The IP -> location table is fictional.
"""
__version__ = "1.0.0"

```

```
"""Correlation engine: turns rule-matched events into alerts.

Three correlation types:

* ``event_count``  - >= threshold matching events per group within ``timespan``
                     (sliding window over sorted timestamps).
* ``value_count``  - >= threshold DISTINCT values of ``field`` per group within
                     ``timespan`` (password spraying: many users from one IP).
* ``impossible_travel`` - consecutive successful logins of a user whose implied
                     speed (Haversine km / hours) exceeds ``max_speed_kmh``.

Once a window crosses the threshold an alert opens; later matching events for the
same group are folded into it while the gap since the alert's last event is within
``timespan``. This yields one alert per burst instead of one per event.
"""
from __future__ import annotations

import hashlib
from collections import Counter, defaultdict, deque
from datetime import timedelta
from typing import Iterable, Optional

from .geo import GeoTable
from .models import Alert, Event
from .rules import Rule

MAX_EVIDENCE = 50
MAX_SAMPLE = 20


def _alert_id(rule_id: str, group: dict, first_seen) -> str:
    key = f"{rule_id}|{sorted(group.items())}|{first_seen.isoformat()}"
    return hashlib.sha256(key.encode()).hexdigest()[:12]


def _group_key(rule: Rule, ev: Event) -> Optional[tuple]:
    vals = tuple(ev.get(f) for f in rule.correlation["group_by"])
    return None if any(v is None for v in vals) else vals


def _new_alert(rule: Rule, key: tuple, events: list[Event]) -> Alert:
    group = dict(zip(rule.correlation["group_by"], key))
    first = events[0].ts
    return Alert(alert_id=_alert_id(rule.rule_id, group, first), rule_id=rule.rule_id,
                 title=rule.title, level=rule.level, label=rule.label, tags=rule.tags,
                 group=group, first_seen=first, last_seen=events[-1].ts,
                 event_count=len(events),
                 evidence=[f"{e.file}:{e.line_no}" for e in events[:MAX_EVIDENCE]])


def _absorb(alert: Alert, ev: Event) -> None:
    alert.last_seen = ev.ts
    alert.event_count += 1
    if len(alert.evidence) < MAX_EVIDENCE:
        alert.evidence.append(f"{ev.file}:{ev.line_no}")


def _grouped(rule: Rule, events: Iterable[Event]) -> dict[tuple, list[Event]]:
    groups: dict[tuple, list[Event]] = defaultdict(list)
    for ev in events:
        if rule.matches(ev):
            key = _group_key(rule, ev)
            if key is not None:
                groups[key].append(ev)
    for evs in groups.values():
```

```
        evs.sort(key=lambda e: e.ts)
    return groups


def detect_event_count(rule: Rule, events: Iterable[Event]) -> list[Alert]:
    span, threshold = rule.timespan, rule.threshold
    alerts: list[Alert] = []
    for key, evs in _grouped(rule, events).items():
        window: deque[Event] = deque()
        active: Optional[Alert] = None
        for ev in evs:
            if active is not None:
                if ev.ts - active.last_seen <= span:
                    _absorb(active, ev)
                    continue
                active = None
            window.append(ev)
            while ev.ts - window[0].ts > span:
                window.popleft()
            if len(window) >= threshold:
                active = _new_alert(rule, key, list(window))
                active.message = (f"{len(window)} matching events from "
                                  f"{', '.join(f'{k}={v}' for k, v in active.group.items())} "
                                  f"within {span}")
                alerts.append(active)
                window.clear()
    for a in alerts:
        a.details["threshold"] = threshold
        a.details["timespan_s"] = span.total_seconds()
        who = ", ".join(f"{k}={v}" for k, v in a.group.items())
        dur = (a.last_seen - a.first_seen).total_seconds()
        a.message = (f"{a.event_count} matching events from {who} over {dur:.0f}s "
                     f"(threshold {threshold:g} within {span})")
    return alerts


def detect_value_count(rule: Rule, events: Iterable[Event]) -> list[Alert]:
    span, threshold = rule.timespan, rule.threshold
    fld = rule.correlation["field"]
    alerts: list[Alert] = []
    for key, evs in _grouped(rule, events).items():
        window: deque[Event] = deque()
        counts: Counter = Counter()
        active: Optional[Alert] = None
        seen: set = set()
        for ev in evs:
            val = ev.get(fld)
            if active is not None:
                if ev.ts - active.last_seen <= span:
                    _absorb(active, ev)
                    if val is not None:
                        seen.add(str(val))
                        active.distinct_count = len(seen)
                        if len(active.distinct_sample) < MAX_SAMPLE and str(val) not in active.dis
tinct_sample:
                            active.distinct_sample.append(str(val))
                    continue
                active = None
            if val is None:
                continue
            window.append(ev)
            counts[val] += 1
            while ev.ts - window[0].ts > span:
                old = window.popleft()
                counts[old.get(fld)] -= 1
```

```
                if counts[old.get(fld)] <= 0:
                    del counts[old.get(fld)]
            if len(counts) >= threshold:
                active = _new_alert(rule, key, list(window))
                seen = {str(v) for v in counts}
                active.distinct_count = len(seen)
                active.distinct_sample = sorted(seen)[:MAX_SAMPLE]
                active.message = (f"{len(seen)} distinct {fld} values from "
                                  f"{', '.join(f'{k}={v}' for k, v in active.group.items())} "
                                  f"within {span}")
                alerts.append(active)
                window.clear()
                counts.clear()
    for a in alerts:
        a.details["threshold"] = threshold
        a.details["timespan_s"] = span.total_seconds()
        who = ", ".join(f"{k}={v}" for k, v in a.group.items())
        dur = (a.last_seen - a.first_seen).total_seconds()
        a.message = (f"{a.distinct_count} distinct {fld} values in {a.event_count} events from {wh
o} "
                     f"over {dur / 60:.0f} min (threshold {threshold:g} within {span})")
    return alerts


def detect_impossible_travel(rule: Rule, events: Iterable[Event], geo: GeoTable,
                             stats: Optional[dict] = None) -> list[Alert]:
    max_speed = float(rule.correlation.get("max_speed_kmh", 900))
    min_dist = float(rule.correlation.get("min_distance_km", 500))
    stats = stats if stats is not None else {}
    stats.setdefault("pairs_checked", 0)
    stats.setdefault("pairs_unknown_geo", 0)
    alerts: list[Alert] = []
    for key, evs in _grouped(rule, events).items():
        for prev, cur in zip(evs, evs[1:]):
            if prev.src_ip == cur.src_ip:
                continue
            stats["pairs_checked"] += 1
            a, b = geo.lookup(prev.src_ip), geo.lookup(cur.src_ip)
            if a is None or b is None:
                stats["pairs_unknown_geo"] += 1
                continue
            dist = geo.distance_km(prev.src_ip, cur.src_ip)
            hours = (cur.ts - prev.ts).total_seconds() / 3600.0
            if dist < min_dist:
                continue
            speed = float("inf") if hours <= 0 else dist / hours
            if speed <= max_speed:
                continue
            alert = _new_alert(rule, key, [prev, cur])
            alert.details = {
                "from_ip": prev.src_ip, "to_ip": cur.src_ip,
                "from_region": a.region_code, "to_region": b.region_code,
                "from_label": a.label, "to_label": b.label,
                "from_lat": a.lat, "from_lon": a.lon, "to_lat": b.lat, "to_lon": b.lon,
                "distance_km": round(dist, 1), "hours": round(hours, 3),
                "speed_kmh": None if speed == float("inf") else round(speed, 1),
                "max_speed_kmh": max_speed, "geo_source": "simulated",
            }
            spd = "instant" if speed == float("inf") else f"{speed:,.0f} km/h"
            alert.message = (f"user {key[0]}: {a.label} -> {b.label}, {dist:,.0f} km in "
                             f"{hours * 60:.0f} min ({spd}; limit {max_speed:,.0f} km/h; simulated
 geo)")
            alerts.append(alert)
    return alerts

```

```
193
194 def run_rules(events: list[Event], rules: list[Rule], geo: Optional[GeoTable] = None,
195                stats: Optional[dict] = None) -> list[Alert]:
196     """Run every rule over the events; returns alerts sorted by first_seen."""
197     geo = geo or GeoTable()
198     alerts: list[Alert] = []
199     for rule in rules:
200         if rule.ctype == "event_count":
201             alerts += detect_event_count(rule, events)
202         elif rule.ctype == "value_count":
203             alerts += detect_value_count(rule, events)
204         elif rule.ctype == "impossible_travel":
205             alerts += detect_impossible_travel(rule, events, geo, stats)
206     alerts.sort(key=lambda a: (a.first_seen, a.rule_id))
207     return alerts
208
```

```
"""Score alerts against the generator's ground-truth labels.

Matching rules (deliberately strict):
* An alert is a TRUE POSITIVE when a *malicious* scenario of the SAME class
  (rule ``label`` == scenario ``type``) involves the same entity and overlaps in time
  (+/- ``tolerance``). Entity = source IP for brute force / spray / HTTP bursts; for
  impossible travel the user must match AND the attacker IP must be one of the two
  logins in the alert.
* Every other alert is a FALSE POSITIVE, including an alert that fires on a real
  attack but under the wrong class (reported separately as ``cross_type``).
* A scenario counts as DETECTED when at least one true-positive alert matched it.

precision = TP alerts / all alerts of that rule;  recall = detected / malicious
scenarios of that class. Denominators are always reported next to the ratios.
"""
from __future__ import annotations

import tempfile
from collections import defaultdict
from datetime import datetime, timedelta
from pathlib import Path
from typing import Iterable, Optional

from .models import Alert

LABELS = ["brute_force", "password_spray", "http_auth_burst", "impossible_travel"]


def _overlap(a0, a1, b0, b1, tol: timedelta) -> bool:
    return a0 <= b1 + tol and b0 - tol <= a1


def _entity_match(alert: Alert, sc: dict) -> bool:
    ent = sc["entity"]
    if alert.label == "impossible_travel":
        ips = {alert.details.get("from_ip"), alert.details.get("to_ip")}
        return alert.group.get("user") == ent.get("user") and ent.get("src_ip") in ips
    return alert.group.get("src_ip") is not None and alert.group.get("src_ip") == ent.get("src_ip"
)


def score(alerts: list[Alert], scenarios: list[dict], tolerance_s: int = 300) -> dict:
    tol = timedelta(seconds=tolerance_s)
    scs = [dict(s, _start=datetime.fromisoformat(s["start"]), _end=datetime.fromisoformat(s["end"]
))
           for s in scenarios]
    detected: set[str] = set()
    per = {lab: {"alerts": 0, "tp_alerts": 0, "fp_alerts": 0, "cross_type": 0, "benign_trap_hits":
 0}
           for lab in LABELS}
    alert_rows = []
    for a in alerts:
        row = per.setdefault(a.label, {"alerts": 0, "tp_alerts": 0, "fp_alerts": 0,
                                       "cross_type": 0, "benign_trap_hits": 0})
        row["alerts"] += 1
        hits = [s for s in scs if _entity_match(a, s)
                and _overlap(a.first_seen, a.last_seen, s["_start"], s["_end"], tol)]
        tp = [s for s in hits if s["malicious"] and s["type"] == a.label]
        verdict = "TP"
        if tp:
            row["tp_alerts"] += 1
            detected.update(s["scenario_id"] for s in tp)
        else:
            row["fp_alerts"] += 1
            verdict = "FP"
```

```
            if any(s["malicious"] for s in hits):
                row["cross_type"] += 1
                verdict = "FP (real attack, other class)"
            elif hits:
                row["benign_trap_hits"] += 1
                verdict = "FP (benign trap)"
        alert_rows.append({"alert_id": a.alert_id, "rule_id": a.rule_id, "verdict": verdict,
                           "matched": [s["scenario_id"] for s in (tp or hits)]})
    for lab in per:
        mal = [s for s in scs if s["malicious"] and s["type"] == lab]
        row = per[lab]
        row["scenarios"] = len(mal)
        row["detected"] = sum(s["scenario_id"] in detected for s in mal)
        for diff in ("normal", "hard"):
            sub = [s for s in mal if s["difficulty"] == diff]
            row[f"scenarios_{diff}"] = len(sub)
            row[f"detected_{diff}"] = sum(s["scenario_id"] in detected for s in sub)
    benign = [s for s in scs if not s["malicious"]]
    trapped = {sid for r in alert_rows if r["verdict"] == "FP (benign trap)" for sid in r["matched
"]}
    missed = [{"scenario_id": s["scenario_id"], "type": s["type"], "difficulty": s["difficulty"],
               "note": s["note"]} for s in scs if s["malicious"] and s["scenario_id"] not in detec
ted]
    return {"per_label": _ratios(per), "alerts": alert_rows, "missed": missed,
            "benign_scenarios": len(benign), "benign_scenarios_alerted": len(trapped)}


def _ratios(per: dict) -> dict:
    for row in per.values():
        row["precision"] = round(row["tp_alerts"] / row["alerts"], 4) if row["alerts"] else None
        row["recall"] = round(row["detected"] / row["scenarios"], 4) if row["scenarios"] else None
        row["recall_normal"] = (round(row["detected_normal"] / row["scenarios_normal"], 4)
                                if row.get("scenarios_normal") else None)
    return per


def aggregate(results: Iterable[dict]) -> dict:
    """Sum per-label counts across several scored datasets and recompute ratios."""
    keys = ["alerts", "tp_alerts", "fp_alerts", "cross_type", "benign_trap_hits", "scenarios",
            "detected", "scenarios_normal", "detected_normal", "scenarios_hard", "detected_hard"]
    tot: dict = defaultdict(lambda: {k: 0 for k in keys})
    benign = benign_hit = n = 0
    for r in results:
        n += 1
        benign += r["benign_scenarios"]
        benign_hit += r["benign_scenarios_alerted"]
        for lab, row in r["per_label"].items():
            for k in keys:
                tot[lab][k] += row.get(k, 0)
    per = _ratios({k: dict(v) for k, v in tot.items()})
    all_alerts = sum(v["alerts"] for v in per.values())
    all_tp = sum(v["tp_alerts"] for v in per.values())
    all_sc = sum(v["scenarios"] for v in per.values())
    all_det = sum(v["detected"] for v in per.values())
    return {"datasets": n, "per_label": per,
            "overall": {"alerts": all_alerts, "tp_alerts": all_tp, "scenarios": all_sc,
                        "detected": all_det,
                        "precision": round(all_tp / all_alerts, 4) if all_alerts else None,
                        "recall": round(all_det / all_sc, 4) if all_sc else None},
            "benign_scenarios": benign, "benign_scenarios_alerted": benign_hit}


def evaluate_seeds(seeds: Iterable[int], rules_dir: str | Path, overrides: Optional[dict] = None,
                   work_dir: Optional[str | Path] = None) -> dict:
    """Generate a fresh dataset per seed, analyse it with the given rules, score it."""
```

```
    from .generator import generate
    from .pipeline import analyze_dir

    seeds = list(seeds)
    results = []
    with tempfile.TemporaryDirectory(prefix="soc_eval_") as tmp:
        base = Path(work_dir or tmp)
        for seed in seeds:
            d = base / f"seed_{seed}"
            labels = generate(d, seed=seed)
            run = analyze_dir(d / "logs", rules_dir, tz=labels["meta"]["auth_tz"],
                              year=labels["meta"]["year"], overrides=overrides)
            results.append(score(run["alerts"], labels["scenarios"]))
    agg = aggregate(results)
    agg["seeds"] = [seeds[0], seeds[-1]] if seeds else []
    return agg

```

```
"""Seeded generator of labelled sample logs (auth.log, syslog, Apache access.log).

Everything here is SYNTHETIC:
* All IPs come from the RFC 5737 documentation ranges (192.0.2.0/24,
  198.51.100.0/24, 203.0.113.0/24) and the usernames are fictional.
* Attack scenarios only reproduce what such activity LOOKS LIKE in a defender's
  logs (timestamps, counts, usernames, status codes). Nothing here sends traffic
  or contains attack tooling.
* Geography comes from the fictional table in data/simulated_geo.csv.

Each run writes ``logs/auth.log`` (BSD syslog, host-local time, no year/zone),
``logs/syslog`` (ISO-8601 with offset), ``logs/access.log`` (Apache combined, web
server in a different zone) and ``labels.json`` / ``labels.csv`` with every
scenario's ground truth, so detections can be scored. The same seed always
produces byte-identical files.
"""
from __future__ import annotations

import csv
import ipaddress
import json
import random
from dataclasses import dataclass, field
from datetime import datetime, timedelta, timezone
from pathlib import Path
from typing import Optional
from zoneinfo import ZoneInfo

MONTH_ABBR = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]

USERS = ["aarav", "priya", "rohan", "sneha", "vikram", "ananya", "kiran", "meera",
         "arjun", "divya", "rahul", "kavya", "nikhil", "pooja", "siddharth", "isha",
         "varun", "neha", "aditya", "riya", "manoj", "lakshmi", "tarun", "swathi"]
COMMON_NAMES = ["root", "admin", "test", "ubuntu", "oracle", "postgres", "guest", "ftpuser",
                "user", "support", "git", "pi", "backup", "mysql", "dev", "jenkins"]

# Fictional regions (must agree with data/simulated_geo.csv).
BLOCKS = {
    "SIM-IN-HYD": "192.0.2.0/27", "SIM-IN-BLR": "192.0.2.32/27", "SIM-IN-MUM": "192.0.2.64/27",
    "SIM-IN-DEL": "192.0.2.96/27", "SIM-SG-SIN": "192.0.2.128/27", "SIM-GB-LON": "192.0.2.160/27",
    "SIM-DE-FRA": "192.0.2.192/27", "SIM-US-NYC": "192.0.2.224/27",
    "SIM-BR-SAO": "198.51.100.32/27", "SIM-AU-SYD": "198.51.100.64/27",
    "SIM-JP-TYO": "198.51.100.96/27", "SIM-ZA-JNB": "198.51.100.128/27",
    "SIM-NL-AMS": "198.51.100.160/27", "SIM-US-SFO": "198.51.100.192/27",
    "SIM-XX-EAST": "203.0.113.0/26", "SIM-XX-WEST": "203.0.113.64/26",
    "SIM-XX-SOUTH": "203.0.113.128/26",
}
OFFICE_NAT = ["198.51.100.10", "198.51.100.11"]     # Simulated Hyderabad office
VPN_EGRESS = "203.0.113.241"                         # allowlisted corporate VPN
MONITOR_IP = "198.51.100.5"
CRAWLER_IP = "198.51.100.200"
FAR_REGIONS = ["SIM-BR-SAO", "SIM-XX-EAST", "SIM-AU-SYD", "SIM-XX-SOUTH", "SIM-US-SFO"]
WEB_PAGES = ["/", "/products", "/products/42", "/about", "/blog", "/blog/post-7", "/pricing",
             "/static/app.js", "/static/style.css", "/contact", "/docs", "/docs/install"]
UAS = ["Mozilla/5.0 (Windows NT 10.0; Win64; x64)", "Mozilla/5.0 (Macintosh; Intel Mac OS X 14_5)"
,
       "Mozilla/5.0 (Linux; Android 14)", "Mozilla/5.0 (iPhone; CPU iPhone OS 17_5)"]


@dataclass
class GenConfig:
    seed: int = 42
    start: datetime = datetime(2026, 10, 19, 18, 30, tzinfo=timezone.utc)  # 00:00 IST Oct 20
    hours: int = 24
    auth_tz: str = "Asia/Kolkata"
```

```
    web_tz: str = "America/New_York"
    ssh_host: str = "bastion-01"
    hard_fraction: float = 0.25     # share of attacks generated below default thresholds


@dataclass
class _Rec:
    ts: datetime
    file: str
    text: str


@dataclass
class Generator:
    cfg: GenConfig
    rng: random.Random = field(init=False)
    recs: list[_Rec] = field(default_factory=list)
    scenarios: list[dict] = field(default_factory=list)
    _pid: int = 2000

    def __post_init__(self):
        self.rng = random.Random(self.cfg.seed)
        self.auth_tz = ZoneInfo(self.cfg.auth_tz)
        self.web_tz = ZoneInfo(self.cfg.web_tz)
        self.end = self.cfg.start + timedelta(hours=self.cfg.hours)
        self._used_ips: set[str] = set()

    # ------------------------------------------------------------------ helpers
    def ip_in(self, region: str, unique: bool = False) -> str:
        net = ipaddress.ip_network(BLOCKS[region])
        hosts = [str(h) for h in net.hosts()]
        for _ in range(100):
            ip = self.rng.choice(hosts)
            if not unique or ip not in self._used_ips:
                self._used_ips.add(ip)
                return ip
        return ip

    def t_at(self, hour_ist: float) -> datetime:
        return self.cfg.start + timedelta(hours=hour_ist)

    def pid(self) -> int:
        self._pid += self.rng.randint(1, 37)
        return self._pid

    def port(self) -> int:
        return self.rng.randint(32768, 60999)

    def auth(self, ts: datetime, msg: str, prog: str = "sshd", pid: Optional[int] = None) -> None:
        lt = ts.astimezone(self.auth_tz)
        p = pid if pid is not None else self.pid()
        head = f"{MONTH_ABBR[lt.month - 1]} {lt.day:>2} {lt:%H:%M:%S} {self.cfg.ssh_host}"
        self.recs.append(_Rec(ts, "auth.log", f"{head} {prog}[{p}]: {msg}"))

    def syslog(self, ts: datetime, prog: str, msg: str, host: str = "app-01") -> None:
        lt = ts.astimezone(self.auth_tz)
        stamp = lt.isoformat(timespec="microseconds")
        self.recs.append(_Rec(ts, "syslog", f"{stamp} {host} {prog}[{self.pid()}]: {msg}"))

    def web(self, ts: datetime, ip: str, method: str, path: str, status: int,
            user: Optional[str] = None, ua: Optional[str] = None, size: Optional[int] = None) -> N
one:
        lt = ts.astimezone(self.web_tz)
        off = lt.strftime("%z")
        stamp = f"{lt.day:02d}/{MONTH_ABBR[lt.month - 1]}/{lt.year}:{lt:%H:%M:%S} {off}"
```

```
        size = size if size is not None else self.rng.randint(180, 24000)
        ua = ua or self.rng.choice(UAS)
        self.recs.append(_Rec(ts, "access.log",
                              f'{ip} - {user or "-"} [{stamp}] "{method} {path} HTTP/1.1" '
                              f'{status} {size} "-" "{ua}"'))

    def ssh_fail(self, ts: datetime, user: str, ip: str, valid_user: bool) -> None:
        pid = self.pid()
        if not valid_user:
            self.auth(ts, f"Invalid user {user} from {ip} port {self.port()}", pid=pid)
        else:
            self.auth(ts, f"pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 "
                          f"tty=ssh ruser= rhost={ip}  user={user}", pid=pid)
        inv = "" if valid_user else "invalid user "
        self.auth(ts + timedelta(milliseconds=self.rng.randint(300, 2500)),
                  f"Failed password for {inv}{user} from {ip} port {self.port()} ssh2", pid=pid)

    def ssh_ok(self, ts: datetime, user: str, ip: str, method: str = "password") -> None:
        pid = self.pid()
        self.auth(ts, f"Accepted {method} for {user} from {ip} port {self.port()} ssh2", pid=pid)
        self.auth(ts + timedelta(milliseconds=40),
                  f"pam_unix(sshd:session): session opened for user {user}(uid=10{USERS.index(user
) if user in USERS else 99:02d}) by (uid=0)",
                  pid=pid)

    def scenario(self, stype: str, malicious: bool, start: datetime, end: datetime,
                 entity: dict, difficulty: str = "normal", note: str = "") -> None:
        self.scenarios.append({
            "scenario_id": f"S{len(self.scenarios) + 1:03d}", "type": stype, "malicious": maliciou
s,
            "difficulty": difficulty, "start": start.isoformat(), "end": end.isoformat(),
            "entity": entity, "note": note})

    # ------------------------------------------------------------------ benign
    def benign(self) -> dict:
        rng = self.rng
        homes: dict[str, str] = {}
        regions = {}
        for i, u in enumerate(USERS):
            # the first ten users live in (simulated) Hyderabad, near the office
            region = "SIM-IN-HYD" if i < 10 else rng.choices(
                ["SIM-IN-HYD", "SIM-IN-BLR", "SIM-IN-MUM", "SIM-IN-DEL"], weights=[4, 3, 2, 2])[0]
            regions[u] = region
            homes[u] = self.ip_in(region, unique=True)
        self.homes, self.regions = homes, regions
        # reserve users for traps / victims so their timelines stay clean
        pool = [u for u in USERS if regions[u] == "SIM-IN-HYD"]
        rng.shuffle(pool)
        self.reserved = {"legit_travel": pool[0], "vpn_switch": pool[1], "typo_burst": pool[2],
                         "web_typos": pool[3]}
        self.victim_pool = [u for u in pool[4:]]
        for u in USERS:
            if u == self.reserved["legit_travel"]:
                continue
            for _ in range(rng.randint(1, 4)):
                h = rng.uniform(8.0, 21.0)
                at_office = regions[u] == "SIM-IN-HYD" and 9 <= h <= 18 and rng.random() < 0.6
                ip = rng.choice(OFFICE_NAT) if at_office else homes[u]
                self.session(u, ip, self.t_at(h), typo_prob=0.12)
        # service account: key-based logins every 30 minutes from the office NAT
        t = self.cfg.start + timedelta(minutes=rng.randint(0, 29))
        while t < self.end:
            self.auth(t, f"Accepted publickey for deploy from {OFFICE_NAT[0]} port {self.port()} s
sh2: "
                         f"ED25519 SHA256:EXAMPLEFAKEFINGERPRINT0000000000000000000000")
```

```
            t += timedelta(minutes=30, seconds=rng.randint(-20, 20))
        # anonymous web visitors from many simulated regions
        for _ in range(rng.randint(140, 180)):
            region = rng.choice(list(BLOCKS))
            ip = self.ip_in(region)
            t = self.t_at(rng.uniform(0, self.cfg.hours - 0.3))
            for _ in range(rng.randint(2, 12)):
                status = rng.choices([200, 304, 404, 401], weights=[80, 10, 8, 2])[0]
                self.web(t, ip, "GET", rng.choice(WEB_PAGES + ["/account"]), status)
                t += timedelta(seconds=rng.uniform(1, 40))
        # syslog noise: cron, systemd, kernel
        t = self.cfg.start
        while t < self.end:
            self.syslog(t, "CRON", "(root) CMD (/usr/local/bin/backup-check --quiet)")
            if rng.random() < 0.3:
                self.syslog(t + timedelta(seconds=7), "systemd",
                            rng.choice(["Started Daily apt download activities.",
                                        "logrotate.service: Deactivated successfully.",
                                        "Starting Cleanup of Temporary Directories..."]))
            if rng.random() < 0.05:
                self.syslog(t + timedelta(seconds=13), "kernel",
                            "[UFW BLOCK] IN=eth0 OUT= SRC=203.0.113.77 DST=192.0.2.250 PROTO=TCP D
PT=23")
            t += timedelta(minutes=5)
        # monitoring health checks every 30 s
        t = self.cfg.start
        while t < self.end:
            self.web(t, MONITOR_IP, "GET", "/healthz", 200, ua="sim-monitor/1.0", size=2)
            t += timedelta(seconds=30)
        return {"users": len(USERS)}

    def session(self, user: str, ip: str, t: datetime, typo_prob: float = 0.0,
                web_login: Optional[bool] = None) -> datetime:
        rng = self.rng
        if rng.random() < typo_prob:
            for _ in range(rng.randint(1, 2)):
                self.ssh_fail(t, user, ip, valid_user=True)
                t += timedelta(seconds=rng.uniform(4, 15))
        self.ssh_ok(t, user, ip, method=rng.choice(["password", "publickey"]))
        if web_login if web_login is not None else rng.random() < 0.7:
            wt = t + timedelta(seconds=rng.uniform(20, 300))
            self.web(wt, ip, "GET", "/login", 200)
            self.web(wt + timedelta(seconds=5), ip, "POST", "/login", 302, user=user)
            wt += timedelta(seconds=6)
            for _ in range(rng.randint(3, 15)):
                self.web(wt, ip, "GET", rng.choice(WEB_PAGES + ["/account", "/dashboard"]),
                         rng.choices([200, 304], weights=[9, 1])[0], user=user)
                wt += timedelta(seconds=rng.uniform(2, 60))
        return t

    # ------------------------------------------------------------------ traps (benign)
    def traps(self) -> None:
        rng, R = self.rng, self.reserved
        # 1) a real user mistypes 6 times in ~2 minutes, then succeeds
        u = R["typo_burst"]
        t = self.t_at(rng.uniform(9, 11))
        t0 = t
        for _ in range(6):
            self.ssh_fail(t, u, self.homes[u], True)
            t += timedelta(seconds=rng.uniform(10, 20))
        self.ssh_ok(t, u, self.homes[u])
        self.scenario("benign_typo_burst", False, t0, t, {"src_ip": self.homes[u], "user": u},
                      note="6 failed passwords then success (below brute-force threshold)")
        # 2) several office users mistype through the same NAT inside 20 minutes
        t = self.t_at(rng.uniform(10, 12))
```

```
        t0 = t
        names = rng.sample([x for x in USERS if self.regions[x] == "SIM-IN-HYD"
                            and x not in R.values()], k=5)
        for name in names:
            self.ssh_fail(t, name, OFFICE_NAT[1], True)
            self.ssh_ok(t + timedelta(seconds=9), name, OFFICE_NAT[1])
            t += timedelta(minutes=rng.uniform(2, 4))
        self.scenario("benign_nat_typos", False, t0, t, {"src_ip": OFFICE_NAT[1]},
                      note="5 distinct users fail once each via shared office NAT")
        # 3) legitimate long-haul travel: Hyderabad -> London 11 h later
        u = R["legit_travel"]
        t1 = self.t_at(rng.uniform(7, 9))
        self.session(u, self.homes[u], t1, web_login=False)
        t2 = t1 + timedelta(hours=11, minutes=rng.randint(0, 50))
        lon = self.ip_in("SIM-GB-LON")
        self.session(u, lon, t2, web_login=True)
        self.scenario("benign_legit_travel", False, t1, t2, {"user": u, "src_ip": lon},
                      note="~7,700 km in 11+ h (< 900 km/h): plausible flight")
        # 4) user switches to the corporate VPN (egress geolocates to New York)
        u = R["vpn_switch"]
        t1 = self.t_at(rng.uniform(12, 14))
        self.session(u, self.homes[u], t1, web_login=False)
        t2 = t1 + timedelta(minutes=rng.randint(10, 20))
        self.session(u, VPN_EGRESS, t2, web_login=True)
        self.scenario("benign_vpn_switch", False, t1, t2, {"user": u, "src_ip": VPN_EGRESS},
                      note="VPN egress is allowlisted in the impossible-travel rule")
        # 5) web user mistypes their password 6 times in under a minute
        u = R["web_typos"]
        ip = self.homes[u]
        t = self.t_at(rng.uniform(14, 16))
        t0 = t
        for _ in range(6):
            self.web(t, ip, "POST", "/login", 401, user=u)
            t += timedelta(seconds=rng.uniform(5, 9))
        self.web(t, ip, "POST", "/login", 302, user=u)
        self.scenario("benign_web_typos", False, t0, t, {"src_ip": ip, "user": u},
                      note="6 x 401 in < 1 min (below 20/60 s)")
        # 6) search-engine-like crawler: lots of requests, 404s and a few 403s
        t = self.t_at(rng.uniform(2, 5))
        t0 = t
        for i in range(250):
            st = rng.choices([200, 404, 403, 301], weights=[70, 22, 4, 4])[0]
            self.web(t, CRAWLER_IP, "GET", f"/page/{i}" if st != 403 else "/private/", st,
                     ua="sim-crawler/2.1 (+https://example.com/bot)")
            t += timedelta(seconds=rng.uniform(1.5, 3.5))
        self.scenario("benign_crawler", False, t0, t, {"src_ip": CRAWLER_IP},
                      note="250 crawler requests, ~4% 403 spread over ~10 min")
        # 7) health check returns 403 during a maintenance blip (filtered by the rule)
        t = self.t_at(rng.uniform(16, 18))
        t0 = t
        for _ in range(60):
            self.web(t, MONITOR_IP, "GET", "/healthz", 403, ua="sim-monitor/1.0", size=9)
            t += timedelta(seconds=0.5)
        self.scenario("benign_healthcheck_403", False, t0, t, {"src_ip": MONITOR_IP},
                      note="60 x 403 on /healthz in 30 s; excluded by filter_healthcheck")
        self.hard_traps()

    def hard_traps(self) -> None:
        """Benign activity that genuinely LOOKS like an attack. Each appears with some
        probability and random intensity, so some datasets contain false-positive bait
        that the default thresholds cannot tell apart from an attack."""
        rng = self.rng
        # a) password-rotation morning: 4-10 distinct office users fail once in ~25 min
        if rng.random() < 0.6:
            k = rng.randint(4, 10)
```

```
            ip = OFFICE_NAT[0]
            t = self.t_at(rng.uniform(9, 10))
            t0 = t
            local = [x for x in USERS if self.regions[x] == "SIM-IN-HYD" and x not in self.reserve
d.values()]
            names = rng.sample(local, k=min(k, len(local)))
            k = len(names)
            for name in names:
                self.ssh_fail(t, name, ip, True)
                self.ssh_ok(t + timedelta(seconds=rng.uniform(6, 20)), name, ip)
                t += timedelta(seconds=rng.uniform(60, 200))
            self.scenario("benign_password_rotation", False, t0, t, {"src_ip": ip},
                          note=f"{k} distinct users mistype once via office NAT (bait for spray ru
le)")
        # b) automation job with an expired password retrying in a loop
        if rng.random() < 0.6:
            ip = OFFICE_NAT[0]
            interval = rng.choice([(15, 45), (90, 150)])
            t = self.t_at(rng.uniform(1, 6))
            t0 = t
            for _ in range(rng.randint(12, 30)):
                self.ssh_fail(t, "svc-backup", ip, True)
                t += timedelta(seconds=rng.uniform(*interval))
            self.scenario("benign_stale_service_account", False, t0, t,
                          {"src_ip": ip, "user": "svc-backup"},
                          note=f"service account retrying an expired password every {interval[0]}-
{interval[1]} s")
        # c) phone tethering: carrier NAT geolocates to (simulated) Singapore
        if rng.random() < 0.5 and self.victim_pool:
            u = self.victim_pool.pop()
            t1 = self.t_at(rng.uniform(10, 12))
            self.session(u, self.homes[u], t1, web_login=False)
            ip = self.ip_in("SIM-SG-SIN", unique=True)
            t2 = t1 + timedelta(minutes=rng.uniform(20, 45))
            self.session(u, ip, t2, web_login=True)
            self.scenario("benign_mobile_carrier", False, t1, t2, {"user": u, "src_ip": ip},
                          note="mobile carrier NAT far from the user (classic impossible-travel FP
)")
        # d) single-page app stuck retrying an expired token
        if rng.random() < 0.5:
            u = rng.choice(USERS)
            ip = self.homes[u]
            t = self.t_at(rng.uniform(12, 20))
            t0 = t
            for _ in range(rng.randint(60, 150)):
                self.web(t, ip, "GET", "/api/notifications", 401, user=u)
                t += timedelta(seconds=rng.uniform(1, 3))
            self.scenario("benign_token_retry_loop", False, t0, t, {"src_ip": ip, "user": u},
                          note="buggy client retrying an expired token (bait for HTTP burst rule)"
)

    # ------------------------------------------------------------------ attacks (labelled)
    def hard(self) -> bool:
        return self.rng.random() < self.cfg.hard_fraction

    def attacks(self) -> None:
        rng = self.rng
        attacker_regions = ["SIM-XX-EAST", "SIM-XX-WEST", "SIM-XX-SOUTH"]
        # SSH brute force against one to three accounts
        for _ in range(rng.randint(2, 3)):
            ip = self.ip_in(rng.choice(attacker_regions), unique=True)
            targets = rng.sample(["root", "admin", "ubuntu", "oracle", rng.choice(USERS)],
                                 k=rng.randint(1, 3))
            hard = self.hard()
            n = rng.randint(14, 18) if hard else rng.randint(40, 150)
```

```
            t = self.t_at(rng.uniform(0.5, 22))
            t0 = t
            for i in range(n):
                user = targets[i % len(targets)] if hard else rng.choice(targets)
                self.ssh_fail(t, user, ip, valid_user=user in USERS)
                t += timedelta(seconds=rng.uniform(240, 420) if hard else rng.uniform(0.5, 4))
            self.scenario("brute_force", True, t0, t, {"src_ip": ip},
                          "hard" if hard else "normal",
                          f"{n} attempts on {targets}" + (" (low-and-slow)" if hard else ""))
        # SSH password spraying: many accounts, one or two tries each
        for _ in range(rng.randint(1, 2)):
            ip = self.ip_in(rng.choice(attacker_regions), unique=True)
            hard = self.hard()
            names = rng.sample(USERS, k=rng.randint(6, 12)) + rng.sample(COMMON_NAMES, k=rng.randi
nt(4, 10))
            rng.shuffle(names)
            t = self.t_at(rng.uniform(0.5, 22))
            t0 = t
            for name in names:
                for _ in range(rng.choice([1, 1, 2])):
                    self.ssh_fail(t, name, ip, valid_user=name in USERS)
                    t += timedelta(seconds=rng.uniform(260, 400) if hard else rng.uniform(20, 80))
            self.scenario("password_spray", True, t0, t, {"src_ip": ip},
                          "hard" if hard else "normal",
                          f"{len(names)} usernames" + (" (slow: >4 min between tries)" if hard els
e ""))
        # HTTP credential stuffing / forced browsing -> 401/403 bursts
        for _ in range(rng.randint(1, 2)):
            ip = self.ip_in(rng.choice(attacker_regions), unique=True)
            hard = self.hard()
            n = rng.randint(30, 50) if hard else rng.randint(40, 200)
            mode = rng.choice(["stuffing", "forced_browsing"])
            t = self.t_at(rng.uniform(0.5, 22))
            t0 = t
            for _ in range(n):
                if mode == "stuffing":
                    self.web(t, ip, "POST", "/login", 401, user=rng.choice(USERS + COMMON_NAMES),
                             ua="python-requests/2.31")
                else:
                    self.web(t, ip, "GET", rng.choice(["/admin", "/admin/config", "/server-status"
,
                                                       "/.git/config", "/backup.zip"]), 403,
                             ua="sim-scanner/0.1")
                t += timedelta(seconds=rng.uniform(4, 6) if hard else rng.uniform(0.2, 1.5))
            self.scenario("http_auth_burst", True, t0, t, {"src_ip": ip},
                          "hard" if hard else "normal", f"{n} x {mode}" + (" (slow)" if hard else
""))
        # Account takeover: victim logs in at home, attacker logs in far away soon after
        victims = rng.sample(self.victim_pool, k=rng.randint(1, 2))
        for u in victims:
            hard = self.hard()
            region = rng.choice(["SIM-DE-FRA", "SIM-GB-LON"] if hard else FAR_REGIONS)
            ip = self.ip_in(region, unique=True)
            t1 = self.t_at(rng.uniform(1, 9))
            self.session(u, self.homes[u], t1, web_login=False)
            gap = timedelta(hours=rng.uniform(10, 13)) if hard else timedelta(minutes=rng.uniform(
15, 120))
            t2 = t1 + gap
            if rng.random() < 0.5:
                self.ssh_ok(t2, u, ip, method="password")
                via = "ssh"
            else:
                self.web(t2, ip, "POST", "/login", 302, user=u, ua="python-requests/2.31")
                via = "web"
            self.scenario("impossible_travel", True, t1, t2, {"user": u, "src_ip": ip},
```

```
                        "hard" if hard else "normal",
                        f"stolen-credential login via {via} from {region} after "
                        f"{gap.total_seconds() / 3600:.1f} h" + (" (physically plausible)" if ha
rd else ""))

    # ---------------------------------------------------------------- invalid lines
    def invalid_lines(self) -> dict[str, list[bytes]]:
        h = self.cfg.ssh_host
        auth = [b"\xff\xfe\x00\x01 binary junk \xc3\x28 not utf-8",
                f"Feb 30 10:00:00 {h} sshd[1]: Failed password for root from 203.0.113.9 port 22 s
sh2".encode(),
                f"Oct 20 10:00:00 {h} sshd[2]: Failed password for root from 999.12.0.1 port 22 ss
h2".encode(),
                b"<<< truncated record",
                f"Oct 20 25:61:00 {h} sshd[3]: Accepted password for x from 192.0.2.1 port 22 ssh2
".encode()]
        sysl = [b"2026-13-40T99:00:00+05:30 app-01 CRON[1]: bad date",
                b"2026-10-20T10:00:00 app-01 CRON[1]: missing timezone",
                b"just some text without a header"]
        web = [b'203.0.113.9 - - [32/Oct/2026:10:00:00 -0400] "GET / HTTP/1.1" 200 12 "-" "x"',
               b'203.0.113.9 - - [20/Oct/2026:10:00:00 -0400] "GET / HTTP/1.1" 999 12 "-" "x"',
               b'300.1.2.3 - - [20/Oct/2026:10:00:00 -0400] "GET / HTTP/1.1" 200 12 "-" "x"',
               b'203.0.113.9 - - [20/Oct/2026:10:00:00 -0400] "BROKEN REQUEST LINE" 400 0',
               b"\x00\x00\x00"]
        pick = lambda pool: self.rng.sample(pool, k=self.rng.randint(2, len(pool)))
        return {"auth.log": pick(auth), "syslog": pick(sysl), "access.log": pick(web)}

    # ---------------------------------------------------------------- output
    def write(self, out_dir: Path) -> dict:
        logs = out_dir / "logs"
        logs.mkdir(parents=True, exist_ok=True)
        bad = self.invalid_lines()
        counts = {}
        for fname in ("auth.log", "syslog", "access.log"):
            recs = sorted((r for r in self.recs if r.file == fname), key=lambda r: (r.ts, r.text))
            lines = [r.text.encode() for r in recs]
            for b in bad[fname]:
                lines.insert(self.rng.randint(1, len(lines) - 1), b)
            (logs / fname).write_bytes(b"\n".join(lines) + b"\n")
            counts[fname] = {"lines": len(lines), "invalid_injected": len(bad[fname])}
        meta = {"seed": self.cfg.seed, "start_utc": self.cfg.start.isoformat(),
                "end_utc": self.end.isoformat(), "auth_tz": self.cfg.auth_tz,
                "web_tz": self.cfg.web_tz, "year": self.cfg.start.astimezone(self.auth_tz).year,
                "files": counts, "generated": True,
                "ip_space": "RFC 5737 documentation ranges only",
                "geo": "fictional table data/simulated_geo.csv"}
        doc = {"meta": meta, "scenarios": self.scenarios}
        (out_dir / "labels.json").write_text(json.dumps(doc, indent=2))
        with open(out_dir / "labels.csv", "w", newline="") as fh:
            wr = csv.writer(fh)
            wr.writerow(["scenario_id", "type", "malicious", "difficulty", "start", "end",
                         "src_ip", "user", "note"])
            for s in self.scenarios:
                wr.writerow([s["scenario_id"], s["type"], s["malicious"], s["difficulty"], s["star
t"],
                             s["end"], s["entity"].get("src_ip", ""), s["entity"].get("user", ""),
                             s["note"]])
        return doc


def generate(out_dir: str | Path, seed: int = 42, **kw) -> dict:
    """Generate one labelled dataset into ``out_dir``; returns the labels document."""
    g = Generator(GenConfig(seed=seed, **kw))
    g.benign()
    g.traps()
```

05-soc-log-detector/src/generator.py (continued)

```
500     g.attacks()
501     return g.write(Path(out_dir))
502
```

```
"""Offline, deterministic, *simulated* IP -> location lookup and Haversine distance.

IMPORTANT: this is not geolocation. Every address used by this project is an
RFC 5737 documentation address (192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24),
which by definition has no real-world location. ``data/simulated_geo.csv`` assigns
fictional regions to sub-blocks so the impossible-travel logic can be exercised
and tested reproducibly. To use real data you would swap this table for a licensed
GeoIP database; the detector only depends on ``GeoTable.lookup``.
"""
from __future__ import annotations

import csv
import ipaddress
import math
from dataclasses import dataclass
from functools import lru_cache
from pathlib import Path
from typing import Optional

EARTH_RADIUS_KM = 6371.0088          # mean Earth radius (IUGG)
DEFAULT_TABLE = Path(__file__).resolve().parent.parent / "data" / "simulated_geo.csv"


@dataclass(frozen=True)
class GeoPoint:
    region_code: str
    label: str
    lat: float
    lon: float


def haversine_km(lat1: float, lon1: float, lat2: float, lon2: float) -> float:
    """Great-circle distance between two (lat, lon) points in kilometres."""
    p1, p2 = math.radians(lat1), math.radians(lat2)
    dp = p2 - p1
    dl = math.radians(lon2 - lon1)
    a = math.sin(dp / 2) ** 2 + math.cos(p1) * math.cos(p2) * math.sin(dl / 2) ** 2
    return 2 * EARTH_RADIUS_KM * math.asin(min(1.0, math.sqrt(a)))


class GeoTable:
    """Longest-prefix-match lookup over a small CIDR table (simulated data)."""

    def __init__(self, path: str | Path = DEFAULT_TABLE):
        self.path = Path(path)
        rows = []
        with open(self.path, newline="", encoding="utf-8") as fh:
            for r in csv.DictReader(fh):
                net = ipaddress.ip_network(r["cidr"], strict=True)
                lat, lon = float(r["lat"]), float(r["lon"])
                if not (-90 <= lat <= 90 and -180 <= lon <= 180):
                    raise ValueError(f"bad coordinates for {r['cidr']}")
                rows.append((net, GeoPoint(r["region_code"], r["label"], lat, lon)))
        # most specific network first => longest prefix wins
        self._rows = sorted(rows, key=lambda x: x[0].prefixlen, reverse=True)
        self.lookup = lru_cache(maxsize=4096)(self._lookup)

    def _lookup(self, ip: Optional[str]) -> Optional[GeoPoint]:
        if not ip:
            return None
        try:
            addr = ipaddress.ip_address(ip)
        except ValueError:
            return None
        for net, point in self._rows:
```

```
            if addr.version == net.version and addr in net:
                return point
        return None

    def distance_km(self, ip_a: str, ip_b: str) -> Optional[float]:
        a, b = self.lookup(ip_a), self.lookup(ip_b)
        if a is None or b is None:
            return None
        return haversine_km(a.lat, a.lon, b.lat, b.lon)

    def __len__(self) -> int:
        return len(self._rows)

```

```python
"""Plain data classes shared by the parser, the rule engine and the reports."""
from __future__ import annotations

from dataclasses import asdict, dataclass, field
from datetime import datetime
from typing import Any, Optional


@dataclass
class Event:
    """One normalised log record. ``ts`` is always timezone-aware UTC."""

    ts: datetime
    source: str                # "auth" | "syslog" | "apache"
    action: str                # e.g. ssh_failed_password, ssh_accepted, http_request
    file: str = ""
    line_no: int = 0
    host: Optional[str] = None
    program: Optional[str] = None
    user: Optional[str] = None
    src_ip: Optional[str] = None
    status: Optional[int] = None     # HTTP status (apache only)
    method: Optional[str] = None
    path: Optional[str] = None
    message: str = ""

    def get(self, name: str) -> Any:
        """Field lookup used by the rule engine (unknown fields -> None)."""
        return getattr(self, name, None)

    def to_dict(self) -> dict:
        d = asdict(self)
        d["ts"] = self.ts.isoformat()
        return d


@dataclass
class InvalidLine:
    """A line the parser refused, kept for data-quality reporting."""

    file: str
    line_no: int
    reason: str
    preview: str  # first 80 printable characters, never the whole line


@dataclass
class Alert:
    """A correlation result produced by one rule."""

    alert_id: str
    rule_id: str
    title: str
    level: str
    label: str                 # scenario class this rule detects (used for evaluation)
    tags: list[str]
    group: dict[str, Any]      # e.g. {"src_ip": "203.0.113.7"} or {"user": "alice"}
    first_seen: datetime
    last_seen: datetime
    event_count: int
    distinct_count: int = 0
    distinct_sample: list[str] = field(default_factory=list)
    evidence: list[str] = field(default_factory=list)   # "file:line" references
    message: str = ""
    details: dict[str, Any] = field(default_factory=dict)
```

```
66
67      def to_dict(self) -> dict:
68          d = asdict(self)
69          d["first_seen"] = self.first_seen.isoformat()
70          d["last_seen"] = self.last_seen.isoformat()
71          return d
72
```

```
"""Defensive parsers for Linux auth.log / syslog and Apache access logs.

Design rules
------------
* Never trust a line: every regex is anchored, IPs are validated with ``ipaddress``,
  timestamps are range-checked, and a line that fails any check becomes an
  ``InvalidLine`` (with a short, sanitised preview) instead of an exception.
* Every timestamp is converted to timezone-aware UTC so events from hosts in
  different time zones can be correlated.
* Classic BSD syslog (RFC 3164, ``Oct 27 08:09:36``) carries no year and no zone.
  The caller supplies both (``tz`` and ``year``); a backwards jump of more than six
  months (December -> January) is treated as a year rollover.
* Lines longer than ``MAX_LINE`` are rejected to bound memory/regex cost.
"""
from __future__ import annotations

import ipaddress
import re
from dataclasses import dataclass, field
from datetime import datetime, timedelta, timezone
from pathlib import Path
from typing import Iterable, Optional
from zoneinfo import ZoneInfo

from .models import Event, InvalidLine

MAX_LINE = 4096
MONTHS = {m: i for i, m in enumerate(
    ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"], 1)}

# --- syslog headers ----------------------------------------------------------
_PROG = r"(?P<prog>[A-Za-z0-9_.\-/]{1,64})(?:\[(?P<pid>\d{1,10})\])?"
BSD_RE = re.compile(
    r"^(?P<mon>[A-Z][a-z]{2}) {1,2}(?P<day>\d{1,2}) (?P<hms>\d{2}:\d{2}:\d{2}) "
    r"(?P<host>[A-Za-z0-9_.\-]{1,255}) " + _PROG + r": (?P<msg>.*)$")
ISO_RE = re.compile(
    r"^(?P<ts>\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}(?:\.\d{1,9})?(?:Z|[+-]\d{2}:\d{2})) "
    r"(?P<host>[A-Za-z0-9_.\-]{1,255}) " + _PROG + r": (?P<msg>.*)$")

# --- sshd message bodies (OpenSSH wording) -----------------------------------
_USER = r"(?P<user>[^\s]{1,64})"
_IP = r"(?P<ip>[0-9A-Fa-f.:]{2,45})"
SSHD_PATTERNS: list[tuple[str, re.Pattern]] = [
    ("ssh_failed_password", re.compile(
        r"^Failed (?:password|publickey|keyboard-interactive/pam) for (?P<invalid>invalid user )?"
        + _USER + r" from " + _IP + r" port \d{1,5}(?: ssh2)?$")),
    ("ssh_accepted", re.compile(
        r"^Accepted (?:password|publickey|keyboard-interactive/pam) for " + _USER
        + r" from " + _IP + r" port \d{1,5}(?: ssh2)?(?:: .*)?$")),
    ("ssh_invalid_user", re.compile(
        r"^Invalid user " + _USER + r" from " + _IP + r"(?: port \d{1,5})?$")),
    ("ssh_disconnect", re.compile(
        r"^(?:Disconnected from|Connection closed by|Received disconnect from)"
        r"(?: (?:invalid|authenticating) user " + _USER + r")? " + _IP + r"(?: port \d{1,5})?.*$")
),
]

# --- Apache combined / common log format -------------------------------------
APACHE_RE = re.compile(
    r'^(?P<ip>\S{2,45}) (?P<ident>\S{1,64}) (?P<user>\S{1,64}) \[(?P<ts>[^\]]{26})\] '
    r'"(?P<method>[A-Z]{3,10}) (?P<path>\S{1,2048}) (?P<proto>HTTP/\d(?:\.\d)?)" '
    r'(?P<status>\d{3}) (?P<size>\d{1,12}|-)(?: "(?P<ref>[^"]{0,2048})" "(?P<ua>[^"]{0,1024})")?$'
)
APACHE_TS_RE = re.compile(
    r"^(?P<d>\d{2})/(?P<mon>[A-Z][a-z]{2})/(?P<y>\d{4}):(?P<H>\d{2}):(?P<M>\d{2}):(?P<S>\d{2}) "
```

```
    r"(?P<sign>[+-])(?P<oh>\d{2})(?P<om>\d{2})$")


class LineError(ValueError):
    """Raised internally when a line is malformed; converted to InvalidLine."""


@dataclass
class ParseResult:
    events: list[Event] = field(default_factory=list)
    invalid: list[InvalidLine] = field(default_factory=list)
    lines_read: int = 0

    def extend(self, other: "ParseResult") -> None:
        self.events.extend(other.events)
        self.invalid.extend(other.invalid)
        self.lines_read += other.lines_read


def _preview(line: str) -> str:
    """Printable, truncated preview so junk/binary lines can't pollute reports."""
    return "".join(ch if ch.isprintable() else "?" for ch in line[:80])


def valid_ip(text: Optional[str]) -> Optional[str]:
    """Return the canonical IP string or raise LineError."""
    if text is None:
        return None
    try:
        return str(ipaddress.ip_address(text))
    except ValueError as exc:
        raise LineError(f"bad_ip") from exc


# ---------------------------------------------------------------------------
class SyslogParser:
    """Parses auth.log / syslog lines in BSD (RFC 3164) or ISO 8601 (RFC 5424-ish,
    rsyslog high-precision) header formats.

    ``tz`` is the zone the host wrote BSD timestamps in; ``year`` is the year of the
    first line. The parser keeps state (last month) for year-rollover inference, so
    use one instance per file, in file order.
    """

    def __init__(self, tz: str = "UTC", year: int = 2026, source: str = "auth"):
        self.tz = ZoneInfo(tz)
        self.year = year
        self.source = source
        self._last_month: Optional[int] = None

    # -- timestamps --
    def _bsd_ts(self, mon: str, day: str, hms: str) -> datetime:
        if mon not in MONTHS:
            raise LineError("bad_month")
        month = MONTHS[mon]
        year = self.year
        if self._last_month is not None and self._last_month - month > 6:
            year += 1                           # Dec -> Jan rollover
        h, m, s = (int(x) for x in hms.split(":"))
        try:
            local = datetime(year, month, int(day), h, m, s, tzinfo=self.tz)
        except ValueError as exc:          # Feb 30, 25:00:00 ...
            raise LineError("bad_timestamp") from exc
        self.year, self._last_month = year, month
        return local.astimezone(timezone.utc)
```

```
129
130     @staticmethod
131     def _iso_ts(text: str) -> datetime:
132         try:
133             dt = datetime.fromisoformat(text.replace("Z", "+00:00"))
134         except ValueError as exc:
135             raise LineError("bad_timestamp") from exc
136         if dt.tzinfo is None:
137             raise LineError("missing_timezone")
138         return dt.astimezone(timezone.utc)
139
140     # -- main entry --
141     def parse_line(self, line: str, file: str = "", line_no: int = 0) -> Event:
142         if len(line) > MAX_LINE:
143             raise LineError("line_too_long")
144         m = BSD_RE.match(line)
145         if m:
146             ts = self._bsd_ts(m["mon"], m["day"], m["hms"])
147         else:
148             m = ISO_RE.match(line)
149             if not m:
150                 raise LineError("unrecognised_format")
151             ts = self._iso_ts(m["ts"])
152         prog, msg = m["prog"], m["msg"]
153         ev = Event(ts=ts, source=self.source, action="syslog", file=file, line_no=line_no,
154                    host=m["host"], program=prog, message=msg[:300])
155         if prog == "sshd":
156             ev.action = "sshd_other"
157             for action, pat in SSHD_PATTERNS:
158                 sm = pat.match(msg)
159                 if sm:
160                     ev.action = action
161                     ev.user = sm.groupdict().get("user")
162                     ev.src_ip = valid_ip(sm["ip"])
163                     break
164         return ev
165
166
167 class ApacheParser:
168     """Parses Apache/Nginx combined (or common) access-log lines.
169
170     The ``%u`` field is taken as the authenticated application user; a POST to a
171     login path returning 2xx/3xx with a user is a ``web_login_success`` and a 401/403
172     on the login path a ``web_login_failure``. Every other line is ``http_request``.
173     """
174
175     LOGIN_PATHS = ("/login", "/api/login", "/wp-login.php", "/user/login")
176
177     def __init__(self, source: str = "apache"):
178         self.source = source
179
180     @staticmethod
181     def _ts(text: str) -> datetime:
182         m = APACHE_TS_RE.match(text)
183         if not m or m["mon"] not in MONTHS:
184             raise LineError("bad_timestamp")
185         offset_min = int(m["oh"]) * 60 + int(m["om"])
186         if offset_min > 14 * 60:
187             raise LineError("bad_timestamp")
188         sign = 1 if m["sign"] == "+" else -1
189         try:
190             tz = timezone(sign * timedelta(minutes=offset_min))
191             dt = datetime(int(m["y"]), MONTHS[m["mon"]], int(m["d"]),
192                           int(m["H"]), int(m["M"]), int(m["S"]), tzinfo=tz)
193         except ValueError as exc:
```

```
            raise LineError("bad_timestamp") from exc
        return dt.astimezone(timezone.utc)

    def parse_line(self, line: str, file: str = "", line_no: int = 0) -> Event:
        if len(line) > MAX_LINE:
            raise LineError("line_too_long")
        m = APACHE_RE.match(line)
        if not m:
            raise LineError("unrecognised_format")
        ip = valid_ip(m["ip"])
        status = int(m["status"])
        if not 100 <= status <= 599:
            raise LineError("bad_status")
        user = None if m["user"] == "-" else m["user"]
        path = m["path"]
        base = path.split("?", 1)[0].lower()
        action = "http_request"
        if base in self.LOGIN_PATHS:
            if m["method"] == "POST" and 200 <= status < 400 and user:
                action = "web_login_success"
            elif status in (401, 403):
                action = "web_login_failure"
        return Event(ts=self._ts(m["ts"]), source=self.source, action=action, file=file,
                     line_no=line_no, user=user, src_ip=ip, status=status,
                     method=m["method"], path=path[:300], message=(m["ua"] or "")[:200])


def parse_lines(lines: Iterable[str], parser, file: str = "") -> ParseResult:
    """Run ``parser`` over lines; malformed lines are collected, never raised."""
    res = ParseResult()
    for n, raw in enumerate(lines, 1):
        res.lines_read += 1
        line = raw.rstrip("\r\n")
        if not line.strip():
            continue                    # blank lines are ignored, not invalid
        try:
            res.events.append(parser.parse_line(line, file=file, line_no=n))
        except LineError as exc:
            res.invalid.append(InvalidLine(file, n, str(exc), _preview(line)))
    return res


def detect_format(sample_lines: list[str]) -> str:
    """Guess 'apache' or 'syslog' from the first non-blank lines."""
    for line in sample_lines:
        line = line.strip()
        if not line:
            continue
        if APACHE_RE.match(line):
            return "apache"
        if BSD_RE.match(line) or ISO_RE.match(line):
            return "syslog"
    return "syslog"


def parse_file(path: str | Path, fmt: str = "auto", tz: str = "UTC", year: int = 2026) -> ParseResult:
    """Parse one file. Bytes that are not UTF-8 are replaced, not fatal."""
    path = Path(path)
    with open(path, "r", encoding="utf-8", errors="replace") as fh:
        lines = fh.readlines()
    if fmt == "auto":
        fmt = detect_format(lines[:50])
    if fmt == "apache":
        parser = ApacheParser()
```

```
    else:
        source = "auth" if "auth" in path.name or "secure" in path.name else "syslog"
        parser = SyslogParser(tz=tz, year=year, source=source)
    return parse_lines(lines, parser, file=path.name)


def parse_paths(paths: Iterable[str | Path], tz: str = "UTC", year: int = 2026,
                tz_overrides: Optional[dict] = None) -> ParseResult:
    """Parse many files; ``tz_overrides`` maps file name -> zone for BSD headers."""
    total = ParseResult()
    for p in paths:
        p = Path(p)
        zone = (tz_overrides or {}).get(p.name, tz)
        total.extend(parse_file(p, tz=zone, year=year))
    total.events.sort(key=lambda e: (e.ts, e.file, e.line_no))
    return total

```

05-soc-log-detector/src/pipeline.py

```python
"""Glue: parse a directory of logs, run the rules, return everything the CLI,
reports and dashboard need."""
from __future__ import annotations

import time
from collections import Counter
from pathlib import Path
from typing import Optional

from .detectors import run_rules
from .geo import GeoTable
from .parsers import parse_paths
from .rules import load_rules

LOG_GLOBS = ("*.log", "syslog", "messages", "secure", "*access*")


def find_logs(log_dir: str | Path) -> list[Path]:
    log_dir = Path(log_dir)
    files: set[Path] = set()
    for g in LOG_GLOBS:
        files.update(p for p in log_dir.glob(g) if p.is_file())
    return sorted(files)


def analyze_dir(log_dir: str | Path, rules_dir: str | Path, tz: str = "UTC", year: int = 2026,
                overrides: Optional[dict] = None, tz_overrides: Optional[dict] = None,
                geo: Optional[GeoTable] = None) -> dict:
    t0 = time.perf_counter()
    files = find_logs(log_dir)
    if not files:
        raise FileNotFoundError(f"no log files found in {log_dir}")
    parsed = parse_paths(files, tz=tz, year=year, tz_overrides=tz_overrides)
    t1 = time.perf_counter()
    rules = load_rules(rules_dir, overrides)
    stats: dict = {}
    alerts = run_rules(parsed.events, rules, geo or GeoTable(), stats)
    t2 = time.perf_counter()
    return {
        "files": [p.name for p in files], "events": parsed.events, "invalid": parsed.invalid,
        "alerts": alerts, "rules": rules, "travel_stats": stats,
        "summary": {
            "lines_read": parsed.lines_read, "events": len(parsed.events),
            "invalid_lines": len(parsed.invalid),
            "events_by_action": dict(Counter(e.action for e in parsed.events).most_common()),
            "events_by_file": dict(Counter(e.file for e in parsed.events)),
            "alerts": len(alerts), "alerts_by_rule": dict(Counter(a.rule_id for a in alerts)),
            "parse_seconds": round(t1 - t0, 4), "detect_seconds": round(t2 - t1, 4),
            "first_event": parsed.events[0].ts.isoformat() if parsed.events else None,
            "last_event": parsed.events[-1].ts.isoformat() if parsed.events else None,
        },
    }

```

```
"""Exports (JSON / CSV), matplotlib charts and a static HTML dashboard."""
from __future__ import annotations

import base64
import csv
import html
import io
import json
from pathlib import Path
from typing import Optional

import matplotlib

matplotlib.use("Agg")
import matplotlib.dates as mdates  # noqa: E402
import matplotlib.pyplot as plt  # noqa: E402
import pandas as pd  # noqa: E402

from .models import Alert, Event, InvalidLine  # noqa: E402

LEVEL_COLORS = {"critical": "#7f0000", "high": "#d62728", "medium": "#ff7f0e",
                "low": "#1f77b4", "informational": "#7f7f7f"}
RULE_COLORS = {"soc-ssh-001": "#d62728", "soc-ssh-002": "#9467bd",
               "soc-web-001": "#ff7f0e", "soc-geo-001": "#2ca02c"}


def write_alerts_json(alerts: list[Alert], summary: dict, path: Path) -> None:
    path.write_text(json.dumps({"summary": summary, "alerts": [a.to_dict() for a in alerts]},
                               indent=2, default=str))


def write_alerts_csv(alerts: list[Alert], path: Path) -> None:
    cols = ["alert_id", "rule_id", "level", "title", "group", "first_seen", "last_seen",
            "event_count", "distinct_count", "message", "tags", "evidence"]
    with open(path, "w", newline="", encoding="utf-8") as fh:
        wr = csv.writer(fh)
        wr.writerow(cols)
        for a in alerts:
            wr.writerow([a.alert_id, a.rule_id, a.level, a.title,
                         ";".join(f"{k}={v}" for k, v in a.group.items()),
                         a.first_seen.isoformat(), a.last_seen.isoformat(), a.event_count,
                         a.distinct_count, a.message, ";".join(a.tags), ";".join(a.evidence[:10])]
)


def write_invalid_csv(invalid: list[InvalidLine], path: Path) -> None:
    with open(path, "w", newline="", encoding="utf-8") as fh:
        wr = csv.writer(fh)
        wr.writerow(["file", "line_no", "reason", "preview"])
        for i in invalid:
            wr.writerow([i.file, i.line_no, i.reason, i.preview])


def events_frame(events: list[Event]) -> pd.DataFrame:
    return pd.DataFrame([{"ts": e.ts, "source": e.source, "action": e.action, "user": e.user,
                          "src_ip": e.src_ip, "status": e.status, "file": e.file,
                          "line_no": e.line_no} for e in events])


def alerts_frame(alerts: list[Alert]) -> pd.DataFrame:
    return pd.DataFrame([{"alert_id": a.alert_id, "rule_id": a.rule_id, "title": a.title,
                          "level": a.level, "label": a.label,
                          "entity": ", ".join(f"{k}={v}" for k, v in a.group.items()),
                          "first_seen": a.first_seen, "last_seen": a.last_seen,
                          "event_count": a.event_count, "message": a.message} for a in alerts])
```

```
65
66
67  def timeline_chart(events: list[Event], alerts: list[Alert], path: Path) -> None:
68      """Event volume per 15 minutes (stacked by source) with alert markers below."""
69      df = events_frame(events)
70      fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(11, 5.6), sharex=True,
71                                     gridspec_kw={"height_ratios": [2.2, 1]})
72      if not df.empty:
73          vol = (df.set_index("ts").groupby("source").resample("15min").size()
74                 .unstack(0).fillna(0))
75          ax1.stackplot(vol.index, *[vol[c] for c in vol.columns], labels=list(vol.columns),
76                        alpha=0.8, colors=["#4c72b0", "#55a868", "#c44e52"][:len(vol.columns)])
77          ax1.legend(loc="upper left", fontsize=8, ncol=3)
78      ax1.set_ylabel("events / 15 min")
79      ax1.set_title("Log volume and alerts over time (UTC) - generated sample data")
80      rules = sorted({a.rule_id for a in alerts})
81      for i, rid in enumerate(rules):
82          for a in (x for x in alerts if x.rule_id == rid):
83              start = a.last_seen if a.label == "impossible_travel" else a.first_seen  # 2nd login
84              end = a.last_seen if a.last_seen - start > pd.Timedelta(minutes=4) else start + pd.Tim
    edelta(minutes=4)
85              ax2.plot([start, end], [i, i], lw=7, solid_capstyle="butt",
86                       color=RULE_COLORS.get(rid, "#333"))
87      ax2.set_yticks(range(len(rules)))
88      ax2.set_yticklabels(rules, fontsize=8)
89      ax2.set_ylim(-0.7, max(len(rules) - 0.3, 0.7))
90      ax2.set_xlabel("time (UTC)")
91      ax2.xaxis.set_major_formatter(mdates.DateFormatter("%m-%d %H:%M"))
92      ax2.grid(axis="x", alpha=0.3)
93      ax1.grid(alpha=0.3)
94      fig.tight_layout()
95      fig.savefig(path, dpi=120)
96      plt.close(fig)
97
98
99  def evaluation_chart(evals: dict[str, dict], path: Path) -> None:
100     """Grouped bars: precision and recall per detector for each threshold preset."""
101     rows = []
102     for preset, ev in evals.items():
103         for lab, r in ev["per_label"].items():
104             rows.append({"preset": preset, "detector": lab, "precision": r["precision"] or 0,
105                          "recall": r["recall"] or 0, "n_alerts": r["alerts"], "n_sc": r["scenarios
    "]})
106     df = pd.DataFrame(rows)
107     labels = list(dict.fromkeys(df["detector"]))
108     presets = list(evals)
109     fig, axes = plt.subplots(1, 2, figsize=(11, 4.2), sharey=True)
110     width = 0.8 / max(len(presets), 1)
111     for ax, metric in zip(axes, ["precision", "recall"]):
112         for j, p in enumerate(presets):
113             sub = df[df.preset == p].set_index("detector").reindex(labels)
114             xs = [i + j * width for i in range(len(labels))]
115             bars = ax.bar(xs, sub[metric], width, label=p)
116             for b, v in zip(bars, sub[metric]):
117                 ax.text(b.get_x() + b.get_width() / 2, v + 0.01, f"{v:.2f}", ha="center", fontsize
    =7)
118         ax.set_xticks([i + width * (len(presets) - 1) / 2 for i in range(len(labels))])
119         ax.set_xticklabels([l.replace("_", "\n") for l in labels], fontsize=8)
120         ax.set_title(metric.capitalize())
121         ax.set_ylim(0, 1.12)
122         ax.grid(axis="y", alpha=0.3)
123     n = next(iter(evals.values()))["datasets"]
124     axes[0].legend(fontsize=8)
125     fig.suptitle(f"Held-out evaluation on {n} generated datasets (labels from the generator)")
126     fig.tight_layout()
```

```
    fig.savefig(path, dpi=120)
    plt.close(fig)


def travel_chart(alerts: list[Alert], path: Path) -> bool:
    """Plot impossible-travel legs on plain lat/lon axes (simulated coordinates)."""
    legs = [a for a in alerts if a.label == "impossible_travel"]
    if not legs:
        return False
    fig, ax = plt.subplots(figsize=(10, 4.8))
    for a in legs:
        d = a.details
        ax.plot([d["from_lon"], d["to_lon"]], [d["from_lat"], d["to_lat"]], "-", color="#2ca02c", alpha=0.6)
        ax.scatter([d["from_lon"], d["to_lon"]], [d["from_lat"], d["to_lat"]], s=25, color="#1f77b4", zorder=3)
        ax.annotate(f'{a.group["user"]}: {d["distance_km"]:,.0f} km', (d["to_lon"], d["to_lat"]),
                    fontsize=7, xytext=(3, 3), textcoords="offset points")
    ax.set_xlim(-180, 180)
    ax.set_ylim(-60, 75)
    ax.set_xlabel("longitude")
    ax.set_ylabel("latitude")
    ax.set_title("Impossible-travel alerts (SIMULATED geo table - RFC 5737 IPs have no real location)")
    ax.grid(alpha=0.3)
    fig.tight_layout()
    fig.savefig(path, dpi=120)
    plt.close(fig)
    return True


def html_dashboard(alerts: list[Alert], summary: dict, timeline_png: Path, path: Path,
                   evaluation: Optional[dict] = None) -> None:
    """Small self-contained HTML report (no JavaScript, no external assets)."""
    img = base64.b64encode(timeline_png.read_bytes()).decode() if timeline_png.exists() else ""
    rows = "\n".join(
        f"<tr><td>{html.escape(a.first_seen.strftime('%Y-%m-%d %H:%M:%S'))}</td>"
        f"<td><span class='lvl' style='background:{LEVEL_COLORS.get(a.level, '#555')}'>{a.level}</span></td>"
        f"<td>{html.escape(a.rule_id)}</td><td>{html.escape(a.title)}</td>"
        f"<td>{html.escape(', '.join(f'{k}={v}' for k, v in a.group.items()))}</td>"
        f"<td>{a.event_count}</td><td>{html.escape(a.message)}</td></tr>" for a in alerts)
    ev_html = ""
    if evaluation:
        ev_rows = "".join(
            f"<tr><td>{lab}</td><td>{r['tp_alerts']}/{r['alerts']}</td><td>{r['precision']}</td>"
            f"<td>{r['detected']}/{r['scenarios']}</td><td>{r['recall']}</td></tr>"
            for lab, r in evaluation["per_label"].items())
        ev_html = (f"<h2>Held-out evaluation ({evaluation['datasets']} generated datasets, default thresholds)</h2>"
                   "<table><tr><th>detector</th><th>TP/alerts</th><th>precision</th>"
                   f"<th>detected/scenarios</th><th>recall</th></tr>{ev_rows}</table>")
    kpis = "".join(f"<div class='kpi'><b>{v}</b><span>{k}</span></div>" for k, v in [
        ("lines read", summary["lines_read"]), ("events parsed", summary["events"]),
        ("invalid lines", summary["invalid_lines"]), ("alerts", summary["alerts"])])
    path.write_text(f"""<!doctype html><html><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>SOC log detector report</title><style>
body{{font-family:system-ui,sans-serif;margin:16px;color:#222}} table{{border-collapse:collapse;width:100%;font-size:13px}}
td,th{{border:1px solid #ddd;padding:4px 6px;text-align:left;vertical-align:top}} th{{background:#f3f3f3}}
.kpi{{display:inline-block;margin:0 12px 12px 0;padding:10px 14px;border-radius:8px;background:#f5f7fb}}
.kpi b{{display:block;font-size:22px}} .kpi span{{font-size:12px;color:#555}}
```

```
.lvl{{color:#fff;border-radius:4px;padding:1px 6px;font-size:12px}} img{{max-width:100%}}
.note{{background:#fff8e1;padding:8px 10px;border-radius:6px;font-size:13px}}</style></head><body>
<h1>SOC log detector report</h1>
<p class="note">All logs are <b>generated</b> sample data; all IPs are RFC 5737 documentation
addresses and the IP-to-location table is <b>fictional</b>. Results say nothing about real-world e
fficacy.</p>
{kpis}<h2>Timeline</h2><img alt="timeline" src="{img}">
<h2>Alerts ({len(alerts)})</h2><table><tr><th>first seen (UTC)</th><th>level</th><th>rule</th><th>
title</th>
<th>entity</th><th>events</th><th>details</th></tr>{rows}</table>{ev_html}
</body></html>""", encoding="utf-8")

```

```
"""Loader and matcher for Sigma-like YAML detection rules.

Supported subset of Sigma (plus a small ``correlation`` block for thresholds):

    detection:
      selection:              # map: all fields must match (AND)
        action: [a, b]        # list: any value matches (OR)
        status|gte: 400       # modifiers: contains startswith endswith re cidr gte lte gt lt
      filter_vpn:
        src_ip|cidr: [203.0.113.240/28]
      condition: selection and not filter_vpn     # and / or / not / parentheses
    correlation:
      type: event_count | value_count | impossible_travel
      group_by: [src_ip]
      timespan: 5m            # s / m / h / d
      threshold: 10
      field: user             # value_count only: distinct values counted

String comparisons are case-insensitive (as in Sigma). Thresholds can be overridden
from config/settings.yml or the CLI (``--set soc-ssh-001.threshold=15``) without
editing rule files.
"""
from __future__ import annotations

import copy
import ipaddress
import re
from dataclasses import dataclass, field
from datetime import timedelta
from pathlib import Path
from typing import Any, Callable, Optional

import yaml

from .models import Event

VALID_TYPES = {"event_count", "value_count", "impossible_travel"}
VALID_LEVELS = {"informational", "low", "medium", "high", "critical"}
_TIMESPAN_RE = re.compile(r"^(\d+)\s*([smhd])$")
_UNITS = {"s": "seconds", "m": "minutes", "h": "hours", "d": "days"}


class RuleError(ValueError):
    """Raised for a rule file that does not follow the supported schema."""


def parse_timespan(text: Any) -> timedelta:
    if isinstance(text, (int, float)):
        return timedelta(seconds=float(text))
    m = _TIMESPAN_RE.match(str(text).strip())
    if not m:
        raise RuleError(f"bad timespan {text!r} (use e.g. 30s, 5m, 1h)")
    return timedelta(**{_UNITS[m[2]]: int(m[1])})


# ---------------------------------------------------------------- field matching
def _norm(v: Any) -> Any:
    return v.lower() if isinstance(v, str) else v


def _compile_field(key: str, expected: Any) -> Callable[[Event], bool]:
    name, *mods = key.split("|")
    values = expected if isinstance(expected, list) else [expected]
    mod = mods[0] if mods else "eq"

```

```
    if mod == "re":
        pats = [re.compile(str(v), re.IGNORECASE) for v in values]
        return lambda e: (v := e.get(name)) is not None and any(p.search(str(v)) for p in pats)
    if mod == "cidr":
        nets = [ipaddress.ip_network(str(v), strict=False) for v in values]

        def _cidr(e: Event) -> bool:
            v = e.get(name)
            if not v:
                return False
            try:
                a = ipaddress.ip_address(v)
            except ValueError:
                return False
            return any(a.version == n.version and a in n for n in nets)
        return _cidr
    if mod in ("gte", "lte", "gt", "lt"):
        import operator
        op = {"gte": operator.ge, "lte": operator.le, "gt": operator.gt, "lt": operator.lt}[mod]
        bound = float(values[0])
        return lambda e: isinstance(e.get(name), (int, float)) and op(e.get(name), bound)
    if mod in ("contains", "startswith", "endswith"):
        vals = [str(v).lower() for v in values]
        fn = {"contains": lambda s, x: x in s, "startswith": str.startswith,
              "endswith": str.endswith}[mod]
        return lambda e: (v := e.get(name)) is not None and any(fn(str(v).lower(), x) for x in val
s)
    if mod == "eq":
        vals = {_norm(v) for v in values}
        return lambda e: _norm(e.get(name)) in vals
    raise RuleError(f"unsupported modifier '{mod}' in '{key}'")


def _compile_selection(sel: Any) -> Callable[[Event], bool]:
    if isinstance(sel, list):                # list of maps => OR
        parts = [_compile_selection(s) for s in sel]
        return lambda e: any(p(e) for p in parts)
    if not isinstance(sel, dict) or not sel:
        raise RuleError("a selection must be a non-empty map or list of maps")
    checks = [_compile_field(k, v) for k, v in sel.items()]
    return lambda e: all(c(e) for c in checks)


# ---------------------------------------------------------------- condition parser
_TOKEN_RE = re.compile(r"\s*(\(|\)|\band\b|\bor\b|\bnot\b|[A-Za-z_][A-Za-z0-9_]*)", re.I)


def _compile_condition(cond: str, selections: dict[str, Callable]) -> Callable[[Event], bool]:
    """Recursive-descent parser: or > and > not > atom."""
    tokens, pos = [], 0
    cond = cond.strip()
    while pos < len(cond):
        m = _TOKEN_RE.match(cond, pos)
        if not m or not m.group(1):
            raise RuleError(f"cannot parse condition near {cond[pos:]!r}")
        tokens.append(m.group(1))
        pos = m.end()
        while pos < len(cond) and cond[pos].isspace():
            pos += 1
    i = 0

    def peek():
        return tokens[i].lower() if i < len(tokens) else None

    def take():
```

```
        nonlocal i
        i += 1
        return tokens[i - 1]

    def expr():
        left = term()
        while peek() == "or":
            take()
            right = term()
            left = (lambda a, b: lambda e: a(e) or b(e))(left, right)
        return left

    def term():
        left = factor()
        while peek() == "and":
            take()
            right = factor()
            left = (lambda a, b: lambda e: a(e) and b(e))(left, right)
        return left

    def factor():
        tok = peek()
        if tok == "not":
            take()
            inner = factor()
            return lambda e: not inner(e)
        if tok == "(":
            take()
            inner = expr()
            if peek() != ")":
                raise RuleError("missing ')' in condition")
            take()
            return inner
        if tok is None:
            raise RuleError("condition ended unexpectedly")
        name = take()
        if name not in selections:
            raise RuleError(f"condition references unknown selection '{name}'")
        return selections[name]

    fn = expr()
    if i != len(tokens):
        raise RuleError(f"unexpected token '{tokens[i]}' in condition")
    return fn


# ---------------------------------------------------------------- rule object
@dataclass
class Rule:
    rule_id: str
    title: str
    level: str
    label: str
    description: str
    tags: list[str]
    correlation: dict[str, Any]
    matcher: Callable[[Event], bool] = field(repr=False)
    path: Optional[str] = None
    falsepositives: list[str] = field(default_factory=list)
    raw: dict = field(default_factory=dict, repr=False)

    @property
    def ctype(self) -> str:
        return self.correlation["type"]

```

```
    @property
    def timespan(self) -> timedelta:
        return parse_timespan(self.correlation.get("timespan", "5m"))

    @property
    def threshold(self) -> float:
        return float(self.correlation.get("threshold", 1))

    def matches(self, ev: Event) -> bool:
        return self.matcher(ev)


def rule_from_dict(doc: dict, path: Optional[str] = None) -> Rule:
    for key in ("title", "id", "level", "detection", "correlation"):
        if key not in doc:
            raise RuleError(f"{path or 'rule'}: missing required key '{key}'")
    level = str(doc["level"]).lower()
    if level not in VALID_LEVELS:
        raise RuleError(f"{path}: invalid level '{level}'")
    det = dict(doc["detection"])
    condition = det.pop("condition", None)
    if not condition:
        raise RuleError(f"{path}: detection.condition is required")
    selections = {name: _compile_selection(sel) for name, sel in det.items()}
    corr = dict(doc["correlation"])
    if corr.get("type") not in VALID_TYPES:
        raise RuleError(f"{path}: correlation.type must be one of {sorted(VALID_TYPES)}")
    if corr["type"] == "value_count" and not corr.get("field"):
        raise RuleError(f"{path}: value_count needs correlation.field")
    corr.setdefault("group_by", ["user"] if corr["type"] == "impossible_travel" else ["src_ip"])
    rule = Rule(rule_id=str(doc["id"]), title=str(doc["title"]), level=level,
                label=str(doc.get("label", doc["id"])),
                description=str(doc.get("description", "")).strip(),
                tags=list(doc.get("tags", [])), correlation=corr,
                matcher=_compile_condition(str(condition), selections), path=path,
                falsepositives=list(doc.get("falsepositives", [])), raw=doc)
    rule.timespan  # validate timespan early
    if rule.threshold <= 0:
        raise RuleError(f"{path}: threshold must be positive")
    return rule


def load_rules(rules_dir: str | Path, overrides: Optional[dict] = None) -> list[Rule]:
    """Load every *.yml/*.yaml rule (safe_load only) and apply threshold overrides."""
    rules = []
    for p in sorted(Path(rules_dir).glob("*.y*ml")):
        with open(p, encoding="utf-8") as fh:
            doc = yaml.safe_load(fh)            # never yaml.load: no object construction
        if not isinstance(doc, dict):
            raise RuleError(f"{p}: not a YAML mapping")
        rules.append(rule_from_dict(doc, str(p)))
    ids = [r.rule_id for r in rules]
    if len(ids) != len(set(ids)):
        raise RuleError("duplicate rule ids")
    return apply_overrides(rules, overrides or {})


def apply_overrides(rules: list[Rule], overrides: dict) -> list[Rule]:
    """``overrides`` = {rule_id: {threshold: 15, timespan: "10m", ...}}; returns copies."""
    known = {r.rule_id for r in rules}
    unknown = set(overrides) - known
    if unknown:
        raise RuleError(f"override for unknown rule id(s): {sorted(unknown)}")
    out = []
    for r in rules:
```

```
        r2 = copy.copy(r)
        r2.correlation = {**r.correlation, **(overrides.get(r.rule_id) or {})}
        r2.timespan
        out.append(r2)
    return out


def parse_set_args(items: list[str]) -> dict:
    """Turn CLI ``rule_id.key=value`` strings into an overrides dict."""
    out: dict = {}
    for item in items or []:
        if "=" not in item or "." not in item.split("=", 1)[0]:
            raise RuleError(f"--set expects rule_id.key=value, got {item!r}")
        lhs, value = item.split("=", 1)
        rid, key = lhs.rsplit(".", 1)
        out.setdefault(rid, {})[key] = yaml.safe_load(value)
    return out

```

05-soc-log-detector/tests/conftest.py

```
from datetime import datetime, timedelta, timezone
from pathlib import Path

import pytest

from src.generator import generate
from src.models import Event

ROOT = Path(__file__).resolve().parent.parent
RULES = ROOT / "rules"
T0 = datetime(2026, 10, 20, 4, 0, tzinfo=timezone.utc)


def ev(sec: float, action="ssh_failed_password", ip="203.0.113.7", user="root", **kw) -> Event:
    """Build a synthetic Event ``sec`` seconds after T0."""
    return Event(ts=T0 + timedelta(seconds=sec), source=kw.pop("source", "auth"), action=action,
                 src_ip=ip, user=user, file="t.log", line_no=int(sec) + 1, **kw)


@pytest.fixture(scope="session")
def sample(tmp_path_factory):
    """One generated dataset (seed 42) shared by the slower tests."""
    d = tmp_path_factory.mktemp("sample")
    labels = generate(d, seed=42)
    return d, labels

```

05-soc-log-detector/tests/test_cli_and_app.py

```
import json
from pathlib import Path

import pytest

import cli

ROOT = Path(__file__).resolve().parent.parent


@pytest.fixture(scope="module")
def generated(tmp_path_factory):
    d = tmp_path_factory.mktemp("cli")
    assert cli.main(["generate", "--seed", "11", "--out", str(d / "s")]) == 0
    return d


def test_generate_and_analyze_write_reports(generated, capsys):
    out = generated / "out"
    assert cli.main(["analyze", "--logs", str(generated / "s" / "logs"), "--out", str(out)]) == 0
    for f in ("alerts.json", "alerts.csv", "invalid_lines.csv", "timeline.png", "dashboard.html"):
        assert (out / f).stat().st_size > 0, f
    doc = json.loads((out / "alerts.json").read_text())
    assert doc["summary"]["alerts"] == len(doc["alerts"]) > 0
    assert all(a["first_seen"].endswith("+00:00") for a in doc["alerts"])
    assert "<script" not in (out / "dashboard.html").read_text()
    assert "alerts:" in capsys.readouterr().out


def test_threshold_override_changes_results(generated):
    base, strict = generated / "b", generated / "st"
    cli.main(["analyze", "--logs", str(generated / "s" / "logs"), "--out", str(base)])
    cli.main(["analyze", "--logs", str(generated / "s" / "logs"), "--out", str(strict),
              "--set", "soc-ssh-001.threshold=1000", "--set", "soc-web-001.threshold=1000"])
    b = json.loads((base / "alerts.json").read_text())["summary"]["alerts_by_rule"]
    s = json.loads((strict / "alerts.json").read_text())["summary"]["alerts_by_rule"]
    assert b.get("soc-ssh-001", 0) > 0 and "soc-ssh-001" not in s and "soc-web-001" not in s


def test_fail_on_alert_and_bad_set_exit_codes(generated):
    logs = str(generated / "s" / "logs")
    assert cli.main(["analyze", "--logs", logs, "--out", str(generated / "f"), "--fail-on-alert"])
 == 1
    assert cli.main(["analyze", "--logs", logs, "--out", str(generated / "g"), "--set", "nope.thre
shold=1"]) == 2
    assert cli.main(["analyze", "--logs", str(generated / "empty"), "--out", str(generated / "h")]
) == 2


def test_streamlit_app_renders_without_errors():
    from streamlit.testing.v1 import AppTest
    at = AppTest.from_file(str(ROOT / "app.py"), default_timeout=120).run()
    assert not at.exception
    assert not at.error, [e.value for e in at.error]
    assert len(at.tabs) == 5
    assert int(at.metric[3].value) > 0           # alerts on the bundled sample

```

05-soc-log-detector/tests/test_evaluate.py

```python
from src.detectors import _new_alert
from src.evaluate import aggregate, evaluate_seeds, score
from src.rules import load_rules
from tests.conftest import RULES, T0, ev


def _alert(rule_id, events):
    rule = {r.rule_id: r for r in load_rules(RULES)}[rule_id]
    key = tuple(events[0].get(f) for f in rule.correlation["group_by"])
    return _new_alert(rule, key, events)


def _sc(sid, stype, mal, ip, start=0, end=120, user=None):
    from datetime import timedelta
    ent = {"src_ip": ip, **({"user": user} if user else {})}
    return {"scenario_id": sid, "type": stype, "malicious": mal, "difficulty": "normal",
            "start": (T0 + timedelta(seconds=start)).isoformat(),
            "end": (T0 + timedelta(seconds=end)).isoformat(), "entity": ent, "note": ""}


def test_tp_fp_cross_type_and_recall():
    tp = _alert("soc-ssh-001", [ev(10, ip="203.0.113.1"), ev(20, ip="203.0.113.1")])
    trap = _alert("soc-ssh-001", [ev(10, ip="198.51.100.10"), ev(20, ip="198.51.100.10")])
    cross = _alert("soc-ssh-002", [ev(10, ip="203.0.113.1"), ev(20, ip="203.0.113.1")])
    scen = [_sc("S1", "brute_force", True, "203.0.113.1"), _sc("S2", "benign_x", False, "198.51.100.10"),
            _sc("S3", "brute_force", True, "203.0.113.99")]
    res = score([tp, trap, cross], scen)
    bf = res["per_label"]["brute_force"]
    assert (bf["alerts"], bf["tp_alerts"], bf["fp_alerts"], bf["benign_trap_hits"]) == (2, 1, 1, 1)
    assert (bf["scenarios"], bf["detected"], bf["precision"], bf["recall"]) == (2, 1, 0.5, 0.5)
    assert res["per_label"]["password_spray"]["cross_type"] == 1
    assert {m["scenario_id"] for m in res["missed"]} == {"S3"}


def test_time_must_overlap():
    late = _alert("soc-ssh-001", [ev(5000, ip="203.0.113.1"), ev(5010, ip="203.0.113.1")])
    res = score([late], [_sc("S1", "brute_force", True, "203.0.113.1", 0, 120)], tolerance_s=300)
    assert res["per_label"]["brute_force"]["tp_alerts"] == 0


def test_aggregate_sums_counts():
    r = {"per_label": {"brute_force": {"alerts": 2, "tp_alerts": 1, "scenarios": 2, "detected": 1}},
         "benign_scenarios": 3, "benign_scenarios_alerted": 1}
    agg = aggregate([r, r])
    assert agg["per_label"]["brute_force"]["precision"] == 0.5
    assert agg["overall"]["alerts"] == 4 and agg["benign_scenarios"] == 6


def test_evaluate_on_two_fresh_seeds_runs():
    res = evaluate_seeds([5001, 5002], RULES)
    assert res["datasets"] == 2 and res["overall"]["scenarios"] > 0
    assert 0 <= res["overall"]["precision"] <= 1 and 0 <= res["overall"]["recall"] <= 1

```

05-soc-log-detector/tests/test_generator_and_fp.py

```python
import ipaddress
import json
import re

from src.evaluate import score
from src.generator import generate
from src.pipeline import analyze_dir
from tests.conftest import RULES

RFC5737 = [ipaddress.ip_network(n) for n in ("192.0.2.0/24", "198.51.100.0/24", "203.0.113.0/24")]
IPV4 = re.compile(rb"\b(\d{1,3}(?:\.\d{1,3}){3})\b")


def test_same_seed_is_byte_identical_and_different_seed_differs(tmp_path):
    a, b, c = tmp_path / "a", tmp_path / "b", tmp_path / "c"
    generate(a, seed=7), generate(b, seed=7), generate(c, seed=8)
    for f in ("auth.log", "syslog", "access.log"):
        assert (a / "logs" / f).read_bytes() == (b / "logs" / f).read_bytes()
    assert (a / "logs" / "auth.log").read_bytes() != (c / "logs" / "auth.log").read_bytes()
    assert (a / "labels.json").read_text() == (b / "labels.json").read_text()


def test_every_valid_ip_in_generated_logs_is_a_documentation_address(sample):
    d, _ = sample
    seen = set()
    for f in (d / "logs").iterdir():
        for raw in IPV4.findall(f.read_bytes()):
            try:
                ip = ipaddress.ip_address(raw.decode())
            except ValueError:
                continue                    # the deliberately malformed 999.x / 300.x lines
            seen.add(ip)
    assert seen and all(any(ip in n for n in RFC5737) for ip in seen)


def test_parser_finds_exactly_the_injected_invalid_lines(sample):
    d, labels = sample
    run = analyze_dir(d / "logs", RULES, tz=labels["meta"]["auth_tz"])
    injected = sum(f["invalid_injected"] for f in labels["meta"]["files"].values())
    assert len(run["invalid"]) == injected
    by_file = {}
    for i in run["invalid"]:
        by_file[i.file] = by_file.get(i.file, 0) + 1
    assert by_file == {f: c["invalid_injected"] for f, c in labels["meta"]["files"].items()}


def test_benign_lookalikes_below_thresholds_do_not_alert(sample):
    """Typos, NAT, a real flight, VPN, a crawler and a filtered health check stay quiet."""
    d, labels = sample
    run = analyze_dir(d / "logs", RULES, tz="Asia/Kolkata")
    sc = score(run["alerts"], labels["scenarios"])
    quiet = {"benign_typo_burst", "benign_nat_typos", "benign_legit_travel", "benign_vpn_switch",
             "benign_web_typos", "benign_crawler", "benign_healthcheck_403"}
    ids = {s["scenario_id"] for s in labels["scenarios"] if s["type"] in quiet}
    assert len(ids) == len(quiet)
    hit = {sid for row in sc["alerts"] for sid in row["matched"] if row["verdict"].startswith("FP")}
    assert not (hit & ids)


def test_wrong_timezone_shifts_events(sample):
    """auth.log has no zone: reading it as UTC instead of IST moves events by 5h30."""
    d, _ = sample
    ist = analyze_dir(d / "logs", RULES, tz="Asia/Kolkata")["events"]
    utc = analyze_dir(d / "logs", RULES, tz="UTC")["events"]
```

```
    a = next(e for e in ist if e.file == "auth.log")
    b = next(e for e in utc if e.file == "auth.log" and e.line_no == a.line_no)
    assert (b.ts - a.ts).total_seconds() == 5.5 * 3600


def test_sample_detects_all_normal_attacks(sample):
    d, labels = sample
    run = analyze_dir(d / "logs", RULES, tz="Asia/Kolkata")
    sc = score(run["alerts"], labels["scenarios"])
    missed = {m["scenario_id"] for m in sc["missed"]}
    normal = {s["scenario_id"] for s in labels["scenarios"] if s["malicious"] and s["difficulty"]
== "normal"}
    assert normal and not (normal & missed)

```

05-soc-log-detector/tests/test_geo.py

```python
import ipaddress
import math

import pytest

from src.detectors import detect_impossible_travel
from src.geo import EARTH_RADIUS_KM, GeoTable, haversine_km
from src.rules import load_rules
from tests.conftest import RULES, ev

RFC5737 = [ipaddress.ip_network(n) for n in ("192.0.2.0/24", "198.51.100.0/24", "203.0.113.0/24")]


def travel_rule(**ov):
    rules = {r.rule_id: r for r in load_rules(RULES, {"soc-geo-001": ov} if ov else None)}
    return rules["soc-geo-001"]


def test_haversine_known_distances():
    assert haversine_km(51.5074, -0.1278, 48.8566, 2.3522) == pytest.approx(343.5, abs=1.5)  # London-Paris
    assert haversine_km(10, 20, 10, 20) == 0
    assert haversine_km(0, 0, 0, 180) == pytest.approx(math.pi * EARTH_RADIUS_KM, rel=1e-9)


def test_geo_table_is_simulated_and_only_documentation_ranges():
    geo = GeoTable()
    for net, point in geo._rows:
        assert any(net.subnet_of(r) for r in RFC5737), net
        assert point.region_code.startswith("SIM-")


def test_longest_prefix_and_unknowns():
    geo = GeoTable()
    assert geo.lookup("203.0.113.241").region_code == "SIM-VPN"
    assert geo.lookup("192.0.2.5").region_code == "SIM-IN-HYD"
    assert geo.lookup("198.51.100.230") is None        # deliberately unmapped
    assert geo.lookup("not-an-ip") is None
    assert geo.lookup("2001:db8::1") is None


def test_fast_far_login_pair_alerts():
    geo = GeoTable()
    evs = [ev(0, action="ssh_accepted", user="priya", ip="192.0.2.5"),            # Hyderabad
           ev(1800, action="ssh_accepted", user="priya", ip="198.51.100.40")]     # Sao Paulo
    alerts = detect_impossible_travel(travel_rule(), evs, geo)
    assert len(alerts) == 1
    d = alerts[0].details
    assert d["distance_km"] > 10000 and d["speed_kmh"] > 20000 and d["geo_source"] == "simulated"


def test_slow_travel_vpn_and_nearby_logins_do_not_alert():
    geo, rule = GeoTable(), travel_rule()
    flight = [ev(0, action="ssh_accepted", user="u", ip="192.0.2.5"),
              ev(11 * 3600, action="ssh_accepted", user="u", ip="192.0.2.170")]       # London, 11 h
    vpn = [ev(0, action="ssh_accepted", user="v", ip="192.0.2.5"),
           ev(600, action="web_login_success", user="v", ip="203.0.113.241", source="apache")]
    office = [ev(0, action="ssh_accepted", user="w", ip="192.0.2.5"),
              ev(300, action="ssh_accepted", user="w", ip="198.51.100.10")]          # same city
    assert detect_impossible_travel(rule, flight + vpn + office, geo) == []


def test_unknown_geo_is_counted_not_alerted_and_threshold_configurable():
    geo, stats = GeoTable(), {}
```

```
    evs = [ev(0, action="ssh_accepted", user="u", ip="192.0.2.5"),
           ev(60, action="ssh_accepted", user="u", ip="198.51.100.230")]
    assert detect_impossible_travel(travel_rule(), evs, geo, stats) == []
    assert stats == {"pairs_checked": 1, "pairs_unknown_geo": 1}
    flight = [ev(0, action="ssh_accepted", user="u", ip="192.0.2.5"),
              ev(11 * 3600, action="ssh_accepted", user="u", ip="192.0.2.170")]
    assert len(detect_impossible_travel(travel_rule(max_speed_kmh=500), flight, geo)) == 1


def test_simultaneous_logins_are_instant_travel():
    evs = [ev(0, action="ssh_accepted", user="u", ip="192.0.2.5"),
           ev(0, action="web_login_success", user="u", ip="198.51.100.70", source="apache")]
    alerts = detect_impossible_travel(travel_rule(), evs, GeoTable())
    assert len(alerts) == 1 and alerts[0].details["speed_kmh"] is None

```

05-soc-log-detector/tests/test_parsers.py

```
from datetime import datetime, timezone

import pytest

from src.parsers import (ApacheParser, LineError, SyslogParser, detect_format, parse_file,
                         parse_lines)


def test_bsd_failed_password_invalid_user_and_timezone():
    p = SyslogParser(tz="Asia/Kolkata", year=2026)
    e = p.parse_line("Oct 20 08:00:05 bastion-01 sshd[4242]: Failed password for invalid user "
                     "admin from 203.0.113.9 port 51234 ssh2")
    assert e.action == "ssh_failed_password"
    assert (e.user, e.src_ip, e.host, e.program) == ("admin", "203.0.113.9", "bastion-01", "sshd")
    # 08:00:05 IST (UTC+05:30) == 02:30:05 UTC
    assert e.ts == datetime(2026, 10, 20, 2, 30, 5, tzinfo=timezone.utc)


def test_bsd_space_padded_day_and_accepted_publickey():
    p = SyslogParser(tz="UTC", year=2026)
    e = p.parse_line("Oct  5 10:00:00 h sshd[1]: Accepted publickey for deploy from 198.51.100.10
"
                     "port 40000 ssh2: ED25519 SHA256:EXAMPLE")
    assert e.action == "ssh_accepted" and e.user == "deploy" and e.ts.day == 5


def test_invalid_user_line_and_non_sshd_program():
    p = SyslogParser()
    assert p.parse_line("Oct 20 10:00:00 h sshd[9]: Invalid user oracle from 203.0.113.5 port 22")
.action == "ssh_invalid_user"
    e = p.parse_line("Oct 20 10:00:01 h CRON[77]: (root) CMD (true)")
    assert e.action == "syslog" and e.program == "CRON" and e.src_ip is None


def test_iso8601_syslog_offset_is_normalised_to_utc():
    p = SyslogParser(tz="UTC", source="syslog")
    e = p.parse_line("2026-10-20T10:00:00.123456+05:30 app-01 systemd[1]: Started x.")
    assert e.ts == datetime(2026, 10, 20, 4, 30, 0, 123456, tzinfo=timezone.utc)


def test_year_rollover_december_to_january():
    p = SyslogParser(tz="UTC", year=2025)
    a = p.parse_line("Dec 31 23:59:59 h CRON[1]: a")
    b = p.parse_line("Jan  1 00:00:01 h CRON[1]: b")
    assert a.ts.year == 2025 and b.ts.year == 2026
    assert (b.ts - a.ts).total_seconds() == 2


def test_failed_rollover_line_does_not_corrupt_year_state():
    p = SyslogParser(tz="UTC", year=2025)
    p.parse_line("Dec 31 23:59:59 h CRON[1]: a")
    with pytest.raises(LineError):
        p.parse_line("Feb 30 00:00:00 h CRON[1]: impossible date")
    assert p.parse_line("Jan  1 00:00:00 h CRON[1]: ok").ts.year == 2026


def test_apache_combined_line_timezone_and_login_classification():
    p = ApacheParser()
    ok = p.parse_line('192.0.2.10 - priya [20/Oct/2026:01:00:00 -0400] "POST /login HTTP/1.1" 302
512 '
                      '"-" "Mozilla/5.0"')
    assert ok.action == "web_login_success" and ok.user == "priya" and ok.status == 302
    assert ok.ts == datetime(2026, 10, 20, 5, 0, tzinfo=timezone.utc)
    bad = p.parse_line('203.0.113.4 - root [20/Oct/2026:01:00:01 -0400] "POST /login HTTP/1.1" 401
 12 "-" "x"')
```

```
    assert bad.action == "web_login_failure"
    plain = p.parse_line('203.0.113.4 - - [20/Oct/2026:01:00:02 +0000] "GET /admin HTTP/1.1" 403 12')
    assert plain.action == "http_request" and plain.user is None and plain.status == 403


@pytest.mark.parametrize("line,reason", [
    ("Oct 20 10:00:00 h sshd[2]: Failed password for root from 999.12.0.1 port 22 ssh2", "bad_ip"),
    ("Feb 30 10:00:00 h sshd[1]: Failed password for root from 203.0.113.9 port 22 ssh2", "bad_timestamp"),
    ("Oct 20 25:61:00 h sshd[3]: Accepted password for x from 192.0.2.1 port 22 ssh2", "bad_timestamp"),
    ("<<< truncated record", "unrecognised_format"),
    ("2026-10-20T10:00:00 app CRON[1]: no zone", "unrecognised_format"),
    ("Oct 20 10:00:00 h CRON[1]: " + "A" * 5000, "line_too_long"),
])
def test_syslog_invalid_lines_are_reported_not_raised(line, reason):
    res = parse_lines([line, ""], SyslogParser(tz="UTC"))
    assert res.events == [] and len(res.invalid) == 1
    assert res.invalid[0].reason == reason
    assert len(res.invalid[0].preview) <= 80


@pytest.mark.parametrize("line,reason", [
    ('203.0.113.9 - - [32/Oct/2026:10:00:00 -0400] "GET / HTTP/1.1" 200 12 "-" "x"', "bad_timestamp"),
    ('203.0.113.9 - - [20/Oct/2026:10:00:00 -0400] "GET / HTTP/1.1" 999 12 "-" "x"', "bad_status"),
    ('300.1.2.3 - - [20/Oct/2026:10:00:00 -0400] "GET / HTTP/1.1" 200 12 "-" "x"', "bad_ip"),
    ('203.0.113.9 - - [20/Oct/2026:10:00:00 -0400] "BROKEN" 400 0', "unrecognised_format"),
])
def test_apache_invalid_lines(line, reason):
    res = parse_lines([line], ApacheParser())
    assert [i.reason for i in res.invalid] == [reason]


def test_non_utf8_bytes_do_not_crash(tmp_path):
    f = tmp_path / "auth.log"
    f.write_bytes(b"\xff\xfe\x00 junk \xc3\x28\n"
                  b"Oct 20 10:00:00 h sshd[1]: Failed password for root from 203.0.113.9 port 22 ssh2\n")
    res = parse_file(f, tz="UTC")
    assert len(res.events) == 1 and len(res.invalid) == 1
    assert res.invalid[0].preview.isprintable()


def test_detect_format():
    assert detect_format(['192.0.2.1 - - [20/Oct/2026:10:00:00 +0000] "GET / HTTP/1.1" 200 1']) == "apache"
    assert detect_format(["", "Oct 20 10:00:00 h CRON[1]: x"]) == "syslog"

```

05-soc-log-detector/tests/test_rules.py

```
import pytest

from src.rules import (RuleError, apply_overrides, load_rules, parse_set_args, parse_timespan,
                       rule_from_dict)
from tests.conftest import RULES, ev


def _rule(detection, corr=None):
    return rule_from_dict({"title": "t", "id": "x", "level": "low", "detection": detection,
                           "correlation": corr or {"type": "event_count", "threshold": 1}})


def test_bundled_rules_load_with_expected_thresholds():
    rules = {r.rule_id: r for r in load_rules(RULES)}
    assert set(rules) == {"soc-ssh-001", "soc-ssh-002", "soc-web-001", "soc-geo-001"}
    assert rules["soc-ssh-001"].threshold == 10 and rules["soc-ssh-001"].timespan.total_seconds()
== 300
    assert rules["soc-ssh-002"].correlation["field"] == "user"
    assert "attack.t1110.003" in rules["soc-ssh-002"].tags


def test_list_values_are_or_and_case_insensitive():
    r = _rule({"sel": {"action": ["SSH_ACCEPTED", "web_login_success"]}, "condition": "sel"})
    assert r.matches(ev(0, action="ssh_accepted"))
    assert not r.matches(ev(0, action="ssh_failed_password"))


@pytest.mark.parametrize("sel,event,expected", [
    ({"path|startswith": "/admin"}, ev(0, path="/admin/x"), True),
    ({"path|contains": "GIT"}, ev(0, path="/.git/config"), True),
    ({"path|endswith": ".zip"}, ev(0, path="/a.php"), False),
    ({"src_ip|cidr": ["203.0.113.240/28"]}, ev(0, ip="203.0.113.241"), True),
    ({"src_ip|cidr": ["203.0.113.240/28"]}, ev(0, ip="203.0.113.7"), False),
    ({"status|gte": 400}, ev(0, status=401), True),
    ({"status|gte": 400}, ev(0, status=None), False),
    ({"user|re": "^svc-"}, ev(0, user="svc-backup"), True),
])
def test_field_modifiers(sel, event, expected):
    assert _rule({"s": sel, "condition": "s"}).matches(event) is expected


def test_condition_and_or_not_parentheses():
    det = {"a": {"action": "ssh_failed_password"}, "b": {"user": "root"}, "c": {"src_ip": "203.0.1
13.1"},
           "condition": "a and not (b or c)"}
    r = _rule(det)
    assert r.matches(ev(0, user="alice", ip="203.0.113.9"))
    assert not r.matches(ev(0, user="root"))
    assert not r.matches(ev(0, user="alice", ip="203.0.113.1"))


@pytest.mark.parametrize("bad", [
    {"s": {"action": "x"}, "condition": "missing"},
    {"s": {"action": "x"}, "condition": "s and"},
    {"s": {"action|nosuchmod": "x"}, "condition": "s"},
    {"s": {}, "condition": "s"},
])
def test_invalid_detection_raises(bad):
    with pytest.raises(RuleError):
        _rule(bad)


def test_timespan_parsing():
    assert parse_timespan("30s").total_seconds() == 30
    assert parse_timespan("2h").total_seconds() == 7200
```

```
    with pytest.raises(RuleError):
        parse_timespan("5 minutes")


def test_overrides_and_cli_set_parsing():
    rules = load_rules(RULES)
    ov = parse_set_args(["soc-ssh-001.threshold=15", "soc-ssh-001.timespan=10m"])
    assert ov == {"soc-ssh-001": {"threshold": 15, "timespan": "10m"}}
    new = {r.rule_id: r for r in apply_overrides(rules, ov)}
    assert new["soc-ssh-001"].threshold == 15 and new["soc-ssh-001"].timespan.total_seconds() == 600
    assert {r.rule_id: r for r in rules}["soc-ssh-001"].threshold == 10   # originals untouched
    with pytest.raises(RuleError):
        apply_overrides(rules, {"no-such-rule": {"threshold": 1}})
    with pytest.raises(RuleError):
        parse_set_args(["threshold=5"])


def test_yaml_object_tags_are_refused(tmp_path):
    (tmp_path / "evil.yml").write_text("!!python/object/apply:os.system ['echo pwned']\n")
    with pytest.raises(Exception) as exc:
        load_rules(tmp_path)
    assert "python/object" in str(exc.value) or "constructor" in str(exc.value).lower()

```

```
from src.detectors import detect_event_count, detect_value_count
from src.rules import rule_from_dict
from tests.conftest import ev


def count_rule(threshold=10, span="5m", group=("src_ip",)):
    return rule_from_dict({"title": "bf", "id": "bf", "level": "high", "label": "brute_force",
                           "detection": {"s": {"action": "ssh_failed_password"}, "condition": "s"},
                           "correlation": {"type": "event_count", "group_by": list(group),
                                           "timespan": span, "threshold": threshold}})


def spray_rule(threshold=8, span="30m"):
    return rule_from_dict({"title": "sp", "id": "sp", "level": "high", "label": "password_spray",
                           "detection": {"s": {"action": "ssh_failed_password"}, "condition": "s"},
                           "correlation": {"type": "value_count", "field": "user", "group_by": ["src_ip"],
                                           "timespan": span, "threshold": threshold}})


def test_below_threshold_no_alert():
    assert detect_event_count(count_rule(), [ev(i * 10) for i in range(9)]) == []


def test_at_threshold_one_alert():
    alerts = detect_event_count(count_rule(), [ev(i * 10) for i in range(10)])
    assert len(alerts) == 1 and alerts[0].event_count == 10
    assert alerts[0].group == {"src_ip": "203.0.113.7"}
    assert len(alerts[0].evidence) == 10


def test_window_edge_is_inclusive_and_spread_out_attempts_do_not_fire():
    # 10 events with the last exactly 300 s after the first -> inside a 5 min window
    assert len(detect_event_count(count_rule(), [ev(i * 300 / 9) for i in range(10)])) == 1
    # 10 events over 6 minutes -> never 10 inside any 5 minute window
    assert detect_event_count(count_rule(), [ev(i * 40) for i in range(10)]) == []


def test_continuing_burst_is_merged_and_separate_bursts_split():
    burst1 = [ev(i * 2) for i in range(50)]                 # 100 s
    burst2 = [ev(3600 + i * 2) for i in range(12)]          # an hour later
    alerts = detect_event_count(count_rule(), burst1 + burst2)
    assert [a.event_count for a in alerts] == [50, 12]


def test_groups_are_independent_and_non_matching_events_ignored():
    evs = [ev(i, ip="203.0.113.1") for i in range(6)] + [ev(i, ip="203.0.113.2") for i in range(6)]
    evs += [ev(i, action="ssh_accepted") for i in range(20)]
    assert detect_event_count(count_rule(threshold=10), evs) == []
    assert len(detect_event_count(count_rule(threshold=6), evs)) == 2


def test_group_by_ip_and_user():
    evs = [ev(i, user="root") for i in range(6)] + [ev(i, user="admin") for i in range(6)]
    assert detect_event_count(count_rule(threshold=10, group=("src_ip", "user")), evs) == []
    assert len(detect_event_count(count_rule(threshold=10, group=("src_ip",)), evs)) == 1


def test_spray_counts_distinct_users_only():
    same_user = [ev(i * 30, user="root") for i in range(40)]
    assert detect_value_count(spray_rule(), same_user) == []
    seven = [ev(i * 60, user=f"u{i}") for i in range(7)]
```

```
    assert detect_value_count(spray_rule(), seven) == []
    eight = [ev(i * 60, user=f"u{i}") for i in range(8)]
    alerts = detect_value_count(spray_rule(), eight)
    assert len(alerts) == 1 and alerts[0].distinct_count == 8
    assert alerts[0].distinct_sample == sorted(f"u{i}" for i in range(8))


def test_spray_window_slides_out_old_users():
    # one new user every 5 minutes: at most 7 distinct users in any 30 minute window
    slow = [ev(i * 300, user=f"u{i}") for i in range(20)]
    assert detect_value_count(spray_rule(), slow) == []

```

05-soc-log-detector/.gitignore

```
__pycache__/
*.pyc
.pytest_cache/
.venv/
venv/
*.db
# private / real logs never belong in the repo
logs_private/
real_logs/
/tmp/

```

05-soc-log-detector/config/sensitive.yml

```
# "Sensitive" preset: lower thresholds / longer windows to catch low-and-slow
# activity. Used by `python cli.py evaluate --overrides config/sensitive.yml`
# to measure the precision/recall trade-off on held-out generated data.
soc-ssh-001: {threshold: 5, timespan: 60m}
soc-ssh-002: {threshold: 5, timespan: 120m}
soc-web-001: {threshold: 10, timespan: 60s}
soc-geo-001: {max_speed_kmh: 700}

```

05-soc-log-detector/config/settings.yml

```
# Analysis settings. Every value can also be given on the CLI.
# Time zone the hosts wrote classic BSD syslog timestamps in (they carry no zone).
auth_log_timezone: Asia/Kolkata
# BSD syslog timestamps carry no year; this is the year of the first line.
log_year: 2026
# Per-file overrides of the BSD time zone (file name -> IANA zone).
timezone_overrides: {}
# Threshold overrides by rule id (applied on top of rules/*.yml).
overrides: {}
#  soc-ssh-001: {threshold: 10, timespan: 5m}

```

05-soc-log-detector/pytest.ini

```
[pytest]
pythonpath = .
testpaths = tests
addopts = -p no:cacheprovider
markers =
    slow: end-to-end tests that generate several datasets
filterwarnings =
    ignore::DeprecationWarning

```

05-soc-log-detector/rules/http_auth_burst.yml

```yaml
title: Burst of HTTP 401/403 responses from one client
id: soc-web-001
status: experimental
level: medium
label: http_auth_burst
description: >
  A single client receives many 401 Unauthorized / 403 Forbidden responses in a
  short window (credential stuffing against a login form, or forced browsing of
  protected paths).
author: Dharma
tags: [attack.credential_access, attack.t1110.004]
logsource: {category: webserver, product: apache}
detection:
  selection:
    source: apache
    status: [401, 403]
  filter_healthcheck:
    path|startswith: /healthz
  condition: selection and not filter_healthcheck
correlation:
  type: event_count
  group_by: [src_ip]
  timespan: 60s
  threshold: 20
falsepositives:
  - A broken client retrying an expired token in a tight loop

```

05-soc-log-detector/rules/impossible_travel.yml

```yaml
title: Impossible travel between two successful logins
id: soc-geo-001
status: experimental
level: high
label: impossible_travel
description: >
  Two consecutive SUCCESSFUL logins for the same user from locations that are too
  far apart for the elapsed time (implied speed above max_speed_kmh). Locations
  come from the simulated offline geo table (data/simulated_geo.csv), not real
  geolocation. Corporate VPN egress is filtered out.
author: Dharma
tags: [attack.initial_access, attack.t1078]
logsource: {product: linux_and_web}
detection:
  selection:
    action: [ssh_accepted, web_login_success]
  filter_vpn:
    src_ip|cidr: [203.0.113.240/28]
  condition: selection and not filter_vpn
correlation:
  type: impossible_travel
  group_by: [user]
  max_speed_kmh: 900
  min_distance_km: 500
falsepositives:
  - Unlisted VPN or cloud egress addresses
  - Mobile carrier NAT that geolocates far from the user

```

05-soc-log-detector/rules/ssh_bruteforce.yml

```yaml
title: SSH brute force from a single source IP
id: soc-ssh-001
status: experimental
level: high
label: brute_force
description: >
  Many failed SSH password attempts from one source IP against the SAME account
  inside a short window (grouped by source IP + user).
  Typical of automated password guessing against one or a few accounts.
author: Dharma
tags: [attack.credential_access, attack.t1110.001]
logsource: {product: linux, service: sshd}
detection:
  selection:
    action: ssh_failed_password
  condition: selection
correlation:
  type: event_count
  group_by: [src_ip, user]
  timespan: 5m
  threshold: 10
falsepositives:
  - A misconfigured automation account retrying an old password
  - Shared NAT egress where many users mistype at once

```

05-soc-log-detector/rules/ssh_password_spray.yml

```yaml
title: SSH password spraying (many usernames from one source)
id: soc-ssh-002
status: experimental
level: high
label: password_spray
description: >
  One source IP fails authentication for many DIFFERENT usernames inside the window
  (few attempts per account, which evades per-account lockout).
author: Dharma
tags: [attack.credential_access, attack.t1110.003]
logsource: {product: linux, service: sshd}
detection:
  selection:
    action: ssh_failed_password
  condition: selection
correlation:
  type: value_count
  field: user
  group_by: [src_ip]
  timespan: 30m
  threshold: 8
falsepositives:
  - Large office NAT where several people mistype passwords in the same half hour

```

```
{
  "meta": {
    "seed": 42,
    "start_utc": "2026-10-19T18:30:00+00:00",
    "end_utc": "2026-10-20T18:30:00+00:00",
    "auth_tz": "Asia/Kolkata",
    "web_tz": "America/New_York",
    "year": 2026,
    "files": {
      "auth.log": {
        "lines": 702,
        "invalid_injected": 2
      },
      "syslog": {
        "lines": 402,
        "invalid_injected": 3
      },
      "access.log": {
        "lines": 4926,
        "invalid_injected": 4
      }
    },
    "generated": true,
    "ip_space": "RFC 5737 documentation ranges only",
    "geo": "fictional table data/simulated_geo.csv"
  },
  "scenarios": [
    {
      "scenario_id": "S001",
      "type": "benign_typo_burst",
      "malicious": false,
      "difficulty": "normal",
      "start": "2026-10-20T03:46:30.438908+00:00",
      "end": "2026-10-20T03:47:59.174899+00:00",
      "entity": {
        "src_ip": "192.0.2.24",
        "user": "sneha"
      },
      "note": "6 failed passwords then success (below brute-force threshold)"
    },
    {
      "scenario_id": "S002",
      "type": "benign_nat_typos",
      "malicious": false,
      "difficulty": "normal",
      "start": "2026-10-20T05:10:37.097897+00:00",
      "end": "2026-10-20T05:24:28.248662+00:00",
      "entity": {
        "src_ip": "198.51.100.11"
      },
      "note": "5 distinct users fail once each via shared office NAT"
    },
    {
      "scenario_id": "S003",
      "type": "benign_legit_travel",
      "malicious": false,
      "difficulty": "normal",
      "start": "2026-10-20T02:53:14.826136+00:00",
      "end": "2026-10-20T14:34:14.826136+00:00",
      "entity": {
        "user": "manoj",
        "src_ip": "192.0.2.183"
      },
      "note": "~7,700 km in 11+ h (< 900 km/h): plausible flight"
    },
```

```
    {
      "scenario_id": "S004",
      "type": "benign_vpn_switch",
      "malicious": false,
      "difficulty": "normal",
      "start": "2026-10-20T07:28:51.188341+00:00",
      "end": "2026-10-20T07:41:51.188341+00:00",
      "entity": {
        "user": "siddharth",
        "src_ip": "203.0.113.241"
      },
      "note": "VPN egress is allowlisted in the impossible-travel rule"
    },
    {
      "scenario_id": "S005",
      "type": "benign_web_typos",
      "malicious": false,
      "difficulty": "normal",
      "start": "2026-10-20T09:57:23.957388+00:00",
      "end": "2026-10-20T09:58:07.206180+00:00",
      "entity": {
        "src_ip": "192.0.2.8",
        "user": "ananya"
      },
      "note": "6 x 401 in < 1 min (below 20/60 s)"
    },
    {
      "scenario_id": "S006",
      "type": "benign_crawler",
      "malicious": false,
      "difficulty": "normal",
      "start": "2026-10-19T20:31:42.572663+00:00",
      "end": "2026-10-19T20:42:05.054768+00:00",
      "entity": {
        "src_ip": "198.51.100.200"
      },
      "note": "250 crawler requests, ~4% 403 spread over ~10 min"
    },
    {
      "scenario_id": "S007",
      "type": "benign_healthcheck_403",
      "malicious": false,
      "difficulty": "normal",
      "start": "2026-10-20T10:59:22.680854+00:00",
      "end": "2026-10-20T10:59:52.680854+00:00",
      "entity": {
        "src_ip": "198.51.100.5"
      },
      "note": "60 x 403 on /healthz in 30 s; excluded by filter_healthcheck"
    },
    {
      "scenario_id": "S008",
      "type": "benign_stale_service_account",
      "malicious": false,
      "difficulty": "normal",
      "start": "2026-10-19T20:57:11.890803+00:00",
      "end": "2026-10-19T21:04:24.815260+00:00",
      "entity": {
        "src_ip": "198.51.100.10",
        "user": "svc-backup"
      },
      "note": "service account retrying an expired password every 15-45 s"
    },
    {
      "scenario_id": "S009",
```

```
      "type": "benign_token_retry_loop",
      "malicious": false,
      "difficulty": "normal",
      "start": "2026-10-20T07:48:58.558502+00:00",
      "end": "2026-10-20T07:52:25.158210+00:00",
      "entity": {
        "src_ip": "192.0.2.18",
        "user": "divya"
      },
      "note": "buggy client retrying an expired token (bait for HTTP burst rule)"
    },
    {
      "scenario_id": "S010",
      "type": "brute_force",
      "malicious": true,
      "difficulty": "normal",
      "start": "2026-10-20T09:39:57.010757+00:00",
      "end": "2026-10-20T09:43:14.089993+00:00",
      "entity": {
        "src_ip": "203.0.113.142"
      },
      "note": "93 attempts on ['ubuntu', 'root', 'sneha']"
    },
    {
      "scenario_id": "S011",
      "type": "brute_force",
      "malicious": true,
      "difficulty": "normal",
      "start": "2026-10-20T00:35:45.016077+00:00",
      "end": "2026-10-20T00:39:44.714422+00:00",
      "entity": {
        "src_ip": "203.0.113.20"
      },
      "note": "109 attempts on ['admin', 'nikhil']"
    },
    {
      "scenario_id": "S012",
      "type": "password_spray",
      "malicious": true,
      "difficulty": "normal",
      "start": "2026-10-20T01:07:22.820050+00:00",
      "end": "2026-10-20T01:27:14.668657+00:00",
      "entity": {
        "src_ip": "203.0.113.166"
      },
      "note": "17 usernames"
    },
    {
      "scenario_id": "S013",
      "type": "http_auth_burst",
      "malicious": true,
      "difficulty": "normal",
      "start": "2026-10-20T15:33:38.821809+00:00",
      "end": "2026-10-20T15:34:50.451479+00:00",
      "entity": {
        "src_ip": "203.0.113.43"
      },
      "note": "85 x forced_browsing"
    },
    {
      "scenario_id": "S014",
      "type": "http_auth_burst",
      "malicious": true,
      "difficulty": "hard",
      "start": "2026-10-20T02:24:59.706303+00:00",
```

```
      "end": "2026-10-20T02:28:20.263643+00:00",
      "entity": {
        "src_ip": "203.0.113.167"
      },
      "note": "41 x stuffing (slow)"
    },
    {
      "scenario_id": "S015",
      "type": "impossible_travel",
      "malicious": true,
      "difficulty": "normal",
      "start": "2026-10-19T20:43:31.922416+00:00",
      "end": "2026-10-19T22:26:06.528327+00:00",
      "entity": {
        "user": "rahul",
        "src_ip": "198.51.100.215"
      },
      "note": "stolen-credential login via ssh from SIM-US-SFO after 1.7 h"
    }
  ]
}
```

05-soc-log-detector/scripts/run_demo.sh

```
#!/usr/bin/env bash
# Reproduce every artifact in outputs/ from scratch (about 15 seconds).
set -euo pipefail
cd "$(dirname "$0")/.."
PY=${PY:-python}
echo '$ python cli.py generate --seed 42 --out samples'
$PY cli.py generate --seed 42 --out samples
echo
echo '$ python cli.py rules'
$PY cli.py rules
echo
echo '$ python cli.py evaluate --seeds 1001-1030 --compare config/sensitive.yml --out outputs'
$PY cli.py evaluate --seeds 1001-1030 --compare config/sensitive.yml --out outputs
echo
echo '$ python cli.py analyze --logs samples/logs --out outputs'
$PY cli.py analyze --logs samples/logs --out outputs

```